

1 Highlights

2 Information Propagation Algorithm for Exact Reliability Analysis 3 in Directed Acyclic Process Networks

4 Temitope Ohiani, Edoardo Patelli, Caroline Pyke

- 5 • ~~Exact reliability computation for directed acyclic process networks with~~
6 ~~reconvergent dependencies~~ source-to-node reliability for DAGs with
7 reconvergent (diamond) dependence
- 8 • ~~Resolves dependent fork-join structures through conditional decomposition~~
9 ~~on shared upstream nodes~~ Diamond decomposition with supernode
10 reuse avoids repeated conditioning
- 11 • ~~Uses progressive diamond decomposition and supernode replacement~~
12 ~~to avoid repeated dependency evaluation~~ Per-instance cost follows from
13 the diamond conditioning sets and is known in advance
- 14 • ~~Reproduces exact benchmark results and is substantially faster than~~
15 ~~a published message-passing alternative~~ Validated against an exact
16 decision-diagram method on 129 graphs and benchmarks
- 17 • ~~Demonstrates that practical tractability is governed primarily by diamond~~
18 ~~nesting depth rather than network size~~ Propagates interval and probability-box
19 component reliabilities natively

Information Propagation Algorithm for Exact Reliability Analysis in Directed Acyclic Process Networks

Temitope Ohiani^a, Edoardo Patelli^{a,*}, Caroline Pyke^b

^a*Department of Civil and Environmental Engineering, University of Strathclyde, Glasgow, UK*

^b*United Kingdom National Nuclear Laboratory, UK*

Abstract

Reliability analysis in directed acyclic process networks requires computing, for each node, the probability that at least one viable path exists from any source node to that node. Standard analytical reliability formulations become inaccurate or inefficient in the presence of reconvergent path structures, where multiple paths diverge from shared upstream ancestors and reconverge downstream, thereby creating statistical dependencies that violate path independence assumptions.

This paper presents the Information Propagation Algorithm (IPA), an exact message-passing method that resolves these dependencies through conditional enumeration on shared fork ancestors. IPA processes the network in topological order, applying closed-form updates for independent parent signals and progressive diamond decomposition for dependent reconvergent structures. ~~Diamond subgraphs are systematically identified, solved through~~ IPA identifies each diamond subgraph, solves it by conditional probability enumeration, and ~~replaced with statistically equivalent supernodes, thereby avoiding repeated conditioning over previously resolved structures~~ replaces it with a statistically equivalent supernode. IPA is a specialisation of cutset conditioning, and its cost is computed from the decomposition before the propagation is run.

~~Two case studies are used to evaluate the proposed method. On a 16-node~~ IPA also propagates interval-valued and probability-box component reliabilities. ~~For interval inputs the propagated range is exact, and for probability-box inputs the bounds on each node's reliability distribution are guaranteed. IPA is validated against an independent binary decision diagram on a benchmark~~

*Corresponding author

Email addresses: `temitope.ohiani@strath.ac.uk` (Temitope Ohiani),
`edoardo.patelli@strath.ac.uk` (Edoardo Patelli), `caroline.pyke@uknnl.com`
(Caroline Pyke)

grid network, ~~IPA reproduces exact reliability values and achieves a 134-fold reduction in runtime relative to a published message-passing alternative.~~ Analysis of six Pareto-optimal power, gas, transit, and water distribution networks, and a corpus of 129 random and structured directed acyclic graphs. ~~It reproduces the exact node reliabilities in every case. It is then applied to a medical drone logistics network configurations shows that computation time depends primarily on diamond nesting depth rather than network size: shallow-nested networks (depth ≤ 12) are solved in under one second, whereas deeply nested structures exhibit the exponential growth inherent to exact inference in high-treewidth graphs.~~ The results demonstrate that IPA provides ~~an exact and computationally efficient reliability analysis tool for Scotland.~~ The analysis identifies the delivery points whose reachability is least certain under the stated input uncertainty. IPA provides exact reliability analysis for a broad class of multi-source directed acyclic process ~~systems, while also identifying networks, and identifies~~ the structural conditions under which exact inference becomes impractical.

Keywords: Network Reliability, Directed Acyclic Graphs, Message Passing, Reconvergent Structures, Imprecise Probability, Process Networks, Infrastructure Systems

1. Introduction

Many infrastructure and engineered systems operate through the propagation of resources ~~, services, or and~~ information across interconnected components. Examples include electric power networks, water distribution systems, telecommunication networks, transportation systems, and logistics networks. In such systems, functionality depends ~~not only~~ on the reliability of individual components ~~but also and~~ on the existence of viable directed paths through which flow can propagate from source nodes to downstream nodes. In this paper, we refer to this class of systems as *multi-source process networks*. Failures in these systems can have severe operational, economic, environmental, and societal consequences. Their assessment therefore requires metrics that quantify ~~not only component reliability but also the ability of the network~~ component reliability and the network's ability to preserve connectivity and sustain flow. A natural reliability measure in this context is the probability that at least one viable path exists from one or more source nodes to a given target node. This source-to-node reachability reliability

provides a direct measure of the network’s ability to maintain function under component and transmission failures.

Directed acyclic graphs (DAGs) are a natural fit to model and evaluate the performance of resource flow systems. In a DAG, nodes represent system components, processing stages, or transfer locations, while directed edges model permissible flow transitions and dependencies between components. Consider the typical process of transport networks where service at downstream locations may depend on the successful operation of several upstream links, hubs, or transfer stages. Failures at earlier stations can cause disruption and delays in travel and, in some cases, stop service altogether.

~~To mitigate~~ Monitoring this reliability is what allows cascading failure scenarios and service bottlenecks ~~the reliability of the network’s resource flow is an important performance metric to monitor. Reliability is generally defined as the probability that a system performs its intended function over a specified time. In flow networks, the intended function is that the other parts of the network successfully receive information flowing from the sources. Complex systems have to be anticipated and mitigated. Complex systems typically rely on~~ redundancy strategies and backup paths to ~~ensure this flow~~. ~~Reliability in multi-source flow networks is related to the probability that at least one viable operational path exists from source nodes to the other nodes in the network.~~ sustain flow when individual components fail.

The classic approach to exactly solving the reliability problem involves enumerating and combining all possible source-to-node paths. However, this quickly becomes computationally intractable for complex process systems with multiple sources, extensive redundancy, and re-convergent network topologies [1]. In particular, when distinct paths share upstream nodes or edges before reconverging downstream, the resulting path events are no longer independent, and naïve path-combination rules become invalid.

The structure of DAGs suggests an alternative viewpoint based on message passing. ~~Rather than globally enumerating all viable paths, reliability~~ Reliability can be computed locally and recursively by asking, at each node: ~~what is the probability that at least one viable signal reaches this node from any source?~~ the probability that at least one viable signal reaches it from any source. For tree-like or conditionally independent substructures, this can be evaluated efficiently using local update rules. However, reconvergent path structures remain problematic because they induce statistical dependencies between incoming signals. These dependencies must be resolved exactly if message passing is to remain accurate.

122 This paper presents the *Information Propagation Algorithm* (IPA), an
 123 exact message-passing method for reliability analysis in directed acyclic pro-
 124 cess networks. IPA exploits topological ordering to propagate source-to-node
 125 reliability from upstream to downstream nodes. Its central idea is to iden-
 126 tify reconvergent fork-join subgraphs, referred to here as *diamond structures*,
 127 and resolve their dependency explicitly by conditioning on the shared fork
 128 ancestors. Once solved, these diamond subgraphs are replaced with statis-
 129 tically equivalent supernodes, ~~allowing the network to be and the network~~
 130 ~~is progressively simplified without loss of exactness. An earlier version of~~
 131 ~~the method was presented as the Information Propagation Method [2]. This~~
 132 ~~paper adds a formal complexity analysis, a validation against an independent~~
 133 ~~exact method, and the propagation of imprecise component reliabilities.~~

134 The proposed approach ~~present several advantages: it preserves exact~~
 135 ~~reliability computation preserves exact probability propagation~~ in DAGs
 136 with reconvergent structures, ~~avoiding the independence errors that arise~~
 137 ~~from where the standard local update rules; by decomposing introduces an~~
 138 ~~independence error. Decomposing~~ and caching repeated dependency patterns
 139 ~~, it reduces repeated reduces~~ conditional enumeration over already-resolved
 140 substructures; ~~and being based on analytical methods, the algorithm allows~~
 141 ~~to analyse large systems in nearly real-time making it adaptable during design~~
 142 ~~phases where multiple analysis of the system under different configurations~~
 143 ~~need to be tested as well for supporting decision tools of critical systems that~~
 144 ~~required a fast assessment of the systems. The cost on a given network is fixed~~
 145 ~~by the decomposition and known before enumeration begins. The approach~~
 146 ~~also propagates interval-valued and probability-box component reliabilities,~~
 147 ~~so that partial knowledge of the component data is carried through the~~
 148 ~~analysis. These properties suit the method to design-stage studies, where~~
 149 ~~many network configurations are assessed, and to decision problems where~~
 150 ~~the reliability inputs are only partly known.~~

151 1.1. Related Works

152 Early reliability analysis methods are based on relatively simple structural
 153 representations of systems. ~~The Reliability Block Diagram (RBD) [3, 4, 5]~~
 154 ~~method models method [3, 4, 5] models a~~ system as interconnected compo-
 155 nent blocks arranged in series, parallel, or k -out-of- n configurations, whose
 156 reliability can be computed using closed-form expressions. While computa-
 157 tionally efficient and intuitive, RBDs are limited in their ability to represent
 158 complex dependency structures and reconvergent flows. Extensions such as

159 Dynamic Reliability Block Diagrams (DRBD) [6] incorporate time-dependent
160 behaviour and ~~allowing for~~ allow more realistic modelling of component inter-
161 actions and system dependencies, but they still struggle to represent highly
162 interconnected networks with multiple convergent paths.

163 Boolean logic-based methods provide a more flexible framework for mod-
164 elling system failure mechanisms by including event causes and consequences.
165 Fault Tree Analysis (FTA) [7] is a top-down deductive logic tree to identify
166 possible combinations of component failures that could cause specific unde-
167 sirable events. Event Tree Analysis (ETA) is the complementary inductive
168 method where the logic tree instead begins with the initiating event and
169 ~~systematically~~ traces all possible sequences of subsequent successes and fail-
170 ures through branching structures and logic gates [8]. These methods can
171 capture a wider range of logical dependencies than RBDs; however, for large-
172 scale systems they require extensive manual construction and become difficult
173 to maintain. In addition, their computational cost grows rapidly with the
174 size and complexity of the logical expressions, particularly in systems with
175 multiple interacting pathways [9, 7].

176 The main limitations of tree-based methods stem from exponential growth
177 in logic expressions and manual construction requirements for complex sys-
178 tems. Binary Decision Diagrams (BDDs) [10, 11, 12] were adapted from
179 circuit design to improve fault tree models by using variable elimination
180 and path reduction techniques on Boolean functions. In BDDs, Boolean
181 functions are represented as directed acyclic graphs (DAGs) based on Shan-
182 non decomposition, where each non-sink node represents a Boolean vari-
183 able with two outgoing edges corresponding to the variable being true or
184 false [7]. This structure allows BDD-based methods to require less mem-
185 ory and computational time than traditional fault tree analysis [10]. BDD
186 algorithms can handle complex Boolean expressions with thousands of vari-
187 ables ~~while maintaining polynomial space complexity,~~ keeping the diagram's
188 size polynomial for many systems. ~~However, their performance depends~~
189 ~~critically on variable ordering, and determining~~ under a favourable variable
190 ordering. Determining an optimal ordering is itself computationally in-
191 tractable in general [13], and a poor ordering can make the diagram grow
192 exponentially instead. For highly interconnected networks with shared com-
193 ponents and reconvergent ~~fanout structures,~~ optimal variable ordering becomes
194 computationally intractable. ~~BDD construction for networks with diamond~~
195 ~~structures – where multiple independent paths converge fan-out.~~ BDD construction
196 meets the reconvergence problem. Multiple paths converging at common

nodes ~~—suffers from the reconvergence problem, requiring~~ require extensive intermediate node generation ~~that, which~~ can exceed memory limitations even for moderately sized systems. These ~~computational challenges become~~ particularly challenges become severe when modelling common cause failures and ~~complex~~ dependency structures that propagate through ~~multiple interconnected pathways simultaneously~~ many interconnected pathways.

Common cause failures create statistical dependencies that violate the independence assumptions underlying traditional reliability calculations[14]. When multiple components share common failure modes, the resulting dependency patterns cannot be ~~efficiently captured~~ captured efficiently through simple Boolean combinations, ~~representing a fundamental~~. This is a limitation shared by RBDs, FTAs, and BDDs in process systems with convergent network topologies.

To overcome these limitations, path-based and graph-theoretic methods have been developed. These methods ~~contextualise treat~~ the network as a probabilistic graph ~~theory problem and leverage problem and apply~~ probability mathematics and set operations to ~~calculate~~ compute network reliability. Exact methods based on minimal path sets and minimal cut sets ~~[15, 16, 3, 17]~~ [15, 16, 3, 17] identify the smallest collections of components that ~~ensure connectivity or guarantee~~ maintain connectivity or cause system failure. Recursive decomposition techniques ~~further~~ improve efficiency by partitioning the network into smaller subproblems. For example, the cut-based method in ~~[17] proposes simultaneously using~~ [17] uses both the disjoint minimal cut sets and the disjoint minimal path sets during ~~the decomposition process~~ decomposition for networks with low edge reliabilities. Kim and Kang’s method [18] extends the Recursive Decomposition Algorithm through a Logical Expansion approach (LE-RDA) that uses ~~two separate procedures for separate~~ intersection and union ~~logical functions to handle decomposition of operations to decompose~~ general system events with multiple node pairs. Nevertheless, these methods remain computationally expensive for networks with extensive redundancy and reconvergent connectivity, as they still require combinatorial enumeration of path or cut structures.

Simulation methods such as Monte Carlo (MC) have become ~~a widely adopted reliability analysis method for~~ widely adopted for reliability analysis of complex networks [19]. MC algorithms can handle arbitrary network structures and provide confidence intervals, although they can become computationally expensive for large networks requiring many samples to reach acceptable accuracy. Monte Carlo variants such as importance sampling[20]

235 and subset simulation [21] have been proposed to address rare failure events,
 236 though these optimisations often demand more computational costs.

237 More recently, message passing approaches based on probabilistic graphi-
 238 cal models have been explored for network reliability analysis [22, 23]. These
 239 methods infer ~~the reliability of the system~~ system reliability by exploit-
 240 ing the network structure to propagate ~~the local calculations throughout~~
 241 ~~the network~~ local calculations until convergence. ~~Unlike enumeration-based~~
 242 ~~methods, message~~ Message passing scales linearly with network size and
 243 ~~naturally~~ handles complex dependency structures directly. For instance,
 244 ~~method for system reliability such as~~ the directed Probability Propagation
 245 Method (dPrPm) [24] ~~, also allows to handle~~ handles dependencies through
 246 pairwise probability tracking and approximation of joint distributions. While
 247 effective, these approaches can become memory-intensive and computation-
 248 ally demanding as the number of dependent interactions increases. In addi-
 249 tion, convergence is not always guaranteed [25], and approximation quality
 250 depends heavily on network topology and component correlation strength.

251 Exact inference in probabilistic graphical models resolves a dependency
 252 induced by shared ancestry either by conditioning or by clustering. Cutset
 253 conditioning enumerates the states of a set of variables whose removal leaves
 254 the rest of the structure singly connected [23]. The junction-tree algorithm
 255 groups variables into a tree of clusters, and its cost is exponential in the
 256 treewidth of the network [26]. Both are exact, and both are governed by
 257 a structural width of the graph. IPA specialises cutset conditioning to the
 258 source-to-node reachability problem, and its cost is governed by the same
 259 width parameter as junction-tree inference and a well-ordered binary decision
 260 diagram (Section 4.3).

261 A component failure probability is often known only within bounds, from
 262 sparse operational data or expert judgement. Such a value is represented as
 263 an interval, asserting only that the true reliability lies between two bounds, or
 264 as a probability box, a pair of bounding cumulative distribution functions [27]
 265 . Williamson and Downs [28] give the arithmetic on probability boxes under
 266 known or unknown dependence, on the classical Fréchet–Hoeffding bounds for
 267 joint distributions. A binary decision diagram can be extended to interval-valued
 268 inputs, using the same monotonicity IPA relies on [29]. The construction does
 269 not scale. Its result also depends on the variable ordering, so a conservative
 270 envelope over several orderings is needed [30]. IPA propagates interval and
 271 probability-box reliabilities through the same conditioning it applies to point
 272 values, without an ordering or a separate monotonicity check.

Hence, existing methods face a trade-off between generality, computational efficiency, and exactness. Classical analytical methods struggle with complex dependency structures, exact path-based approaches suffer from combinatorial explosion, and simulation or approximate message-passing methods sacrifice exactness for scalability. This work addresses this gap by proposing an exact message-passing framework for DAG reliability analysis that explicitly resolves reconvergent dependencies through conditional decomposition of fork-join substructures. By identifying and reducing these structures during computation, the proposed method avoids repeated enumeration over shared dependencies ~~while preserving and~~ preserves exact probabilistic results.

2. Network Model

A process system ~~is~~ can be modelled as a directed acyclic graph $G = (V, E)$ where V denotes the set of nodes (components) and $E \subseteq V \times V$ represents the set of directed edges (dependencies). Let $S \subseteq V$ denote the ~~sets of source and~~ $T \subseteq V$ ~~the sink~~ set of source nodes. Throughout the paper we use $\text{Pa}(v)$ ~~and~~ $\text{Ch}(v)$ to represent the set of parent nodes ~~and child nodes~~ of v , ~~respectively~~.

2.1. Component States and Probabilistic Model

Each node $v \in V$ is associated with a binary random variable $X_v \in \{0, 1\}$, where $X_v = 1$ indicates that node v is operational. Similarly, each directed edge $(u, v) \in E$ is associated with a binary random variable $X_{uv} \in \{0, 1\}$ representing successful transmission from u to v .

We define:

- $R(v) = P(X_v = 1)$: intrinsic reliability of node v
- $P(u, v) = P(X_{uv} = 1)$: transmission probability of edge (u, v)

A directed path $\pi(s, v) = (s = v_0, v_1, \dots, v_k = v)$ is said to be *viable* if all nodes and edges along the path are operational, i.e.

$$X_{v_0} X_{v_1} \cdots X_{v_k} \cdot X_{v_0 v_1} \cdots X_{v_{k-1} v_k} = 1.$$

Definition 2.1 (Viable Path). *A path $\pi(s, v)$ from source $s \in S$ to node $v \in V$ is viable if all nodes and edges along the path are functional.*

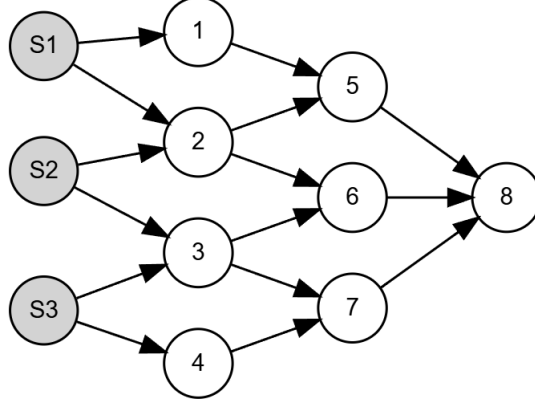


Figure 1: A Multi-Source Directed Acyclic Graph.

303 2.2. Definition of Diamond Structures

304 We formalise reconvergent dependency patterns through the ~~notion~~concept
 305 of a diamond subgraph.

306 **Definition 2.2** (Diamond Structure). *A diamond structure is a subgraph*
 307 *$D \subseteq G$ characterised by:*

- 308 • *a set of fork nodes $F \subseteq V$, where each $f \in F$ has at least two outgoing*
 309 *paths,*
- 310 • *a join node $j \in V$ with at least two incoming paths,*
- 311 • *at least two distinct directed paths from nodes in F to j ,*
- 312 • *at least one shared upstream node or edge among these paths.*

313 In such structures, the paths from fork nodes to the join node are not
 314 independent because they share common components. The simplest case is
 315 a *single-fork diamond*, where all reconvergent paths originate from a single
 316 fork node.

317 2.3. Reliability Objective

318 The reliability problem is formally stated as computing the probability
 319 that there is at least one viable path from any source node to each target
 320 node in the network. The reachability probability of a node $v \in V$ is defined

321 as the probability that at least one viable path exists from any source node
 322 to v :

$$b(v) = P \left(\bigcup_{s \in S} \bigcup_{\pi \in \Pi(s, v)} \{\pi \text{ is viable}\} \right), \quad (1)$$

323 where $\Pi(s, v)$ denotes the set of all directed paths from s to v .

324 The quantity $b(v)$ represents the probability that node v ~~successfully~~-re-
 325 ceives flow from at least one source and is referred to as the *node reliability*
 326 (or belief) at v .

327 The reachability of v can also be written recursively. With X_v and X_{uv}
 328 the node and edge indicators,

$$\mathcal{R}_v = X_v \wedge \left(v \in S \vee \bigvee_{u \in \text{Pa}(v)} (\mathcal{R}_u \wedge X_{uv}) \right), \quad (2)$$

329 and $b(v) = P(\mathcal{R}_v = 1)$. Because the component indicators are mutually independent
 330 (Assumption 2.2), $\mathcal{R}_v$ is a monotone Boolean function of them: restoring a
 331 failed component can create source-to-node connectivity but never remove it.
 332 Computing $b(v)$ is #P-hard in general, since two-terminal network reliability
 333 is a special case [31, 32]. The monotonicity makes $b(v)$ non-decreasing in
 334 every node prior and every edge probability, which Section 2.5 uses to propagate
 335 interval-valued inputs exactly.

336 For a node v , let $\text{anc}(v)$ be its ancestors, the nodes with a directed path to
 337 v . Given a set C of forks whose reachability states are fixed by conditioning,
 338 the *unresolved influence* of a parent u of a join is

$$\text{infl}(u; C) = (\{u\} \cup \text{anc}(u)) \setminus C, \quad (3)$$

339 the upstream components on which the signal through u still depends. Two
 340 parents of a join are statistically dependent when their unresolved influences
 341 intersect. This is the structural test used by the decomposition of Section 4.

342

343 2.4. Model Assumptions

344 **Assumption 2.1** (Graph Structure). *The network G is a directed acyclic*
 345 *graph (i.e. G contains no directed cycles), and every node $v \notin S$ is reachable*
 346 *from at least one source node.*

347 **Assumption 2.2** (Component Independence). *Node states $\{X_v\}$ and edge*
 348 *states $\{X_{uv}\}$ are mutually independent. Node and edge states are also inde-*
 349 *pendent of each other.*

350 **Assumption 2.3** (Binary Operation). *All nodes and edges have binary states*
 351 *(operational or failed).*

352 Dependencies between paths arise solely from shared nodes or edges. In
 353 particular, signals arriving at a node from different parents are independent
 354 only if the corresponding upstream paths do not share common components.

355 This formulation is equivalent to a network reliability problem with node
 356 and edge failures, where the objective is to compute source-to-node reacha-
 357 bility probabilities.

358 2.5. *Imprecise Component Reliabilities*

359 The model above takes the component reliabilities $R(v)$ and $P(u, v)$
 360 as known point values. In many applications these inputs are themselves
 361 uncertain: a failure probability may be estimated from sparse operational
 362 data or elicited from an expert. Each component reliability is therefore
 363 allowed in one of three forms of increasing informativeness:

- 364 1. a point value $p \in [0, 1]$, as above;
- 365 2. an interval $[p, \bar{p}] \subset [0, 1]$, asserting only that the true reliability lies
 366 between the stated bounds;
- 367 3. a probability box: a pair of cumulative distribution functions $(F, \bar{F})$
 368 that bound the distribution of the uncertain reliability, appropriate
 369 when partial distributional information is available [27, 28].

370 The reliability objective generalises with the input. For an interval input, the
 371 target is the exact range $[b(v), \bar{b}(v)]$ of node reliabilities over all component
 372 reliabilities consistent with the stated intervals. For a probability-box input,
 373 the target is a sound pair of bounds on the distribution of $b(v)$. Section 5.3
 374 shows that the propagation scheme computes the interval range exactly, by
 375 the monotonicity established above, and computes guaranteed distributional
 376 bounds in the probability-box case.

377 3. The Information Propagation Algorithm

378 The Information Propagation Algorithm (IPA) computes source-to-node
 379 reliability in a directed acyclic graph by propagating reachability probabilities

380 from upstream nodes to downstream nodes in topological order. For each
 381 node $v \in V$, the objective is to compute

$$b(v) = P(\text{at least one viable path from any source to } v), \quad (4)$$

382 where $b(v)$ denotes the reliability of node v .

383 3.1. Signal Propagation Formulation

384 Let v be a non-source node with parent set

$$\text{Pa}(v) = \{p_1, p_2, \dots, p_k\}.$$

385 For each parent $p_i \in \text{Pa}(v)$, define the incoming signal event

$$A_i = \{\text{a viable signal reaches } v \text{ through parent } p_i\}.$$

386 The corresponding signal probability is

$$S_i = P(A_i) = b(p_i) P(p_i, v), \quad (5)$$

387 that is, the probability that a viable path reaches p_i and is successfully trans-
 388 mitted through edge (p_i, v) .

389 Since node v must itself be operational in order to receive and transmit
 390 flow, its reliability is

$$b(v) = R(v) P\left(\bigcup_{i=1}^k A_i\right). \quad (6)$$

391 For a source node $v_s \in S$, reliability is simply its intrinsic reliability:

$$b(v_s) = R(v_s). \quad (7)$$

392 3.2. Topological Evaluation Strategy

393 Because the network is acyclic, node reliabilities can be evaluated in topo-
 394 logical order. Let

$$L = \{L_1, L_2, \dots, L_K\}$$

395 denote a layering of the graph such that every edge goes from an earlier layer
 396 to a later one. Then, for any node $v \in L_i$, all parent nodes $p \in \text{Pa}(v)$ belong
 397 to layers L_j with $j < i$.

398 This property allows IPA to evaluate all node reliabilities in a single
 399 forward pass:

- 400 • source nodes are initialised using (7);
 - 401 • each non-source node is processed once all parent reliabilities are avail-
402 able;
 - 403 • the local update depends on whether the incoming parent signals are
404 independent or dependent.
- 405 No iterative convergence procedure is required.

406 3.2.1. Independent Parent Signals

407 If the incoming signal events $\{A_i\}_{i=1}^k$ are mutually independent, then the
408 probability that node v receives at least one signal is

$$P\left(\bigcup_{i=1}^k A_i\right) = 1 - \prod_{i=1}^k (1 - S_i). \quad (8)$$

409 Substituting (8) into (6) gives

$$b(v) = R(v) \left[1 - \prod_{p \in \text{Pa}(v)} (1 - b(p)P(p, v)) \right]. \quad (9)$$

410 Equation (9) is exact when the upstream paths associated with the in-
411 coming parent signals do not share common nodes or edges. Two important
412 special cases are shown in Figures 2 and 3.

413 For a node with a single parent p , the update reduces to the chained-path
414 case

$$b(v) = R(v) b(p) P(p, v). \quad (10)$$

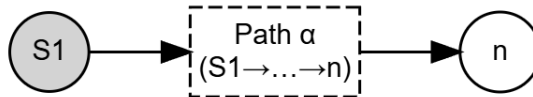


Figure 2: Single-path structure: chained signal propagation.

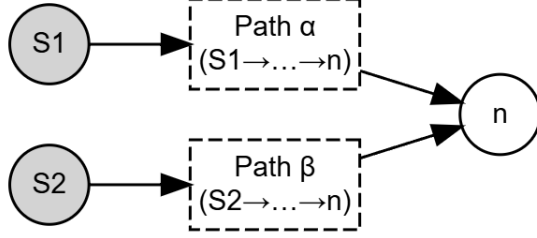


Figure 3: Multiple independent parent signals: parallel signal propagation.

415 3.2.2. Reconvergent Structures and Independence Breakdown

416 The update rule in (9) is no longer exact when the incoming signal
 417 events are dependent. In directed acyclic graphs, this occurs in *reconver-*
 418 *gent structures*, where multiple upstream paths share common components
 419 before merging at a downstream node.

420 Let v be a join node receiving signals from parents $\{p_1, p_2\}$. If the up-
 421 stream paths leading to p_1 and p_2 share a common ancestor f , then the events
 422 A_1 and A_2 are dependent. In that case, applying the independence-based rule

$$P(A_1 \cup A_2) = 1 - (1 - S_1)(1 - S_2) \quad (11)$$

423 overestimates the true probability of signal reception, because it ignores the
 424 correlation induced by the shared upstream components.

425 IPA resolves this problem by identifying such reconvergent structures and
 426 conditioning on the shared upstream variables that induce the dependency.

427 3.2.3. Diamond Structures and Conditional Decomposition

428 The fundamental dependency-inducing pattern considered by IPA is a *di-*
 429 *amond structure*: a reconvergent fork-join subgraph in which multiple paths
 430 diverge from one or more shared upstream nodes and reconverge at a join
 431 node.

432 Consider a single-fork diamond with fork node f and join node j . Let

$$J = \{\text{a signal reaches } j\}, \quad F = \{\text{fork node } f \text{ is operational}\}.$$

433 By the law of total probability,

$$P(J) = P(J | F)P(F) + P(J | \bar{F})P(\bar{F}). \quad (12)$$

434 Conditioning on the state of the fork restores independence among the
 435 downstream paths:

- 436 • if $F = 1$, the fork is active and the paths downstream of f become
437 conditionally independent;
 - 438 • if $F = 0$, no signal can propagate through paths originating from f .
- 439 Thus, if the diamond contains k downstream paths from f to j , then

$$P(J \mid F) = 1 - \prod_{i=1}^k (1 - S_i^{(F)}), \quad (13)$$

440 where $S_i^{(F)}$ denotes the probability that path i transmits a signal to j given
441 that the fork is active.

442 For a simple induced diamond with no alternative upstream routes to the
443 join node,

$$P(J \mid \bar{F}) = 0. \quad (14)$$

444 3.2.4. Generalisation to Multiple Conditioning Nodes

445 More complex reconvergent structures may involve several shared up-
446 stream nodes

$$\mathcal{C} = \{c_1, c_2, \dots, c_m\}.$$

447 In that case, the join-node signal probability is computed by enumerating all
448 conditioning states:

$$P(J) = \sum_{s \in \{0,1\}^m} P(J \mid s) P(s), \quad (15)$$

449 where $s = (s_1, \dots, s_m)$ is a binary assignment to the conditioning nodes and

$$P(s) = \prod_{i=1}^m [b(c_i)]^{s_i} [1 - b(c_i)]^{1-s_i}. \quad (16)$$

450 For each conditioning state s , the remaining incoming paths to the join
451 node become conditionally independent, so the local update can again be
452 evaluated using the independent-signal formula.

453 3.3. Simple Example

454 To illustrate the effect of reconvergent dependencies and the correctness
455 of the conditioning approach, consider the simple diamond in Figure 4. The
456 subgraph consists of a single fork–join pair connected by two intermediate
457 paths. All non-source nodes have intrinsic reliability 0.9, and all edges have
458 transmission probability 0.9.

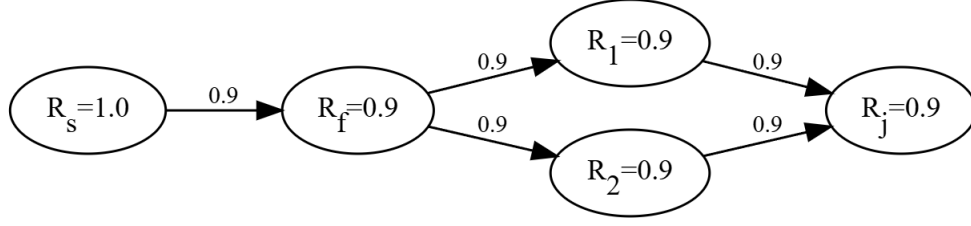


Figure 4: A simple diamond subgraph between fork node f and join node j .

459 3.3.1. Solution by Path Enumeration (Correct but Expensive)

460 We enumerate the relevant fork/intermediate-node states:

$$\begin{aligned}
 P_1 &= 0.81 \times 0.81 \times 0.81 = 0.531441 && \text{(fork active, both intermediates work),} \\
 P_2 &= 0.81 \times 0.81 \times 0.19 = 0.124659 && \text{(fork active, only first intermediate works),} \\
 P_3 &= 0.81 \times 0.19 \times 0.81 = 0.124659 && \text{(fork active, only second intermediate works),} \\
 P_4 &= 0.81 \times 0.19 \times 0.19 = 0.029241 && \text{(fork active, both intermediates fail),} \\
 P_5 &= 0.19 && \text{(fork inactive, no signal possible).}
 \end{aligned}$$

461 The signal-reception probabilities at the join node are

$$P(\text{signal}) = \begin{cases} 0.99, & \text{if both paths send signals,} \\ 0.9, & \text{if only one path sends a signal,} \\ 0, & \text{if no path sends a signal.} \end{cases}$$

462 Hence,

$$\begin{aligned}
 P(\text{signal}) &= P_1 \times 0.99 + P_2 \times 0.9 + P_3 \times 0.9 \\
 &= 0.531441 \times 0.99 + 0.124659 \times 0.9 + 0.124659 \times 0.9 \\
 &= 0.75051279.
 \end{aligned}$$

463 The resulting reliability at the join node is

$$b(j) = R(j) P(\text{signal}) = 0.9 \times 0.75051279 = 0.675461511.$$

464 3.3.2. Solution by Independent Update (Incorrect)

465 Using the independent-parent update yields

$$\begin{aligned}
 b(f) &= R(s) P(s, f) R(f) = 1.0 \times 0.9 \times 0.9 = 0.81, \\
 b(p_1) &= b(p_2) = b(f) P(f, p) R(p) = 0.81 \times 0.9 \times 0.9 = 0.6561.
 \end{aligned}$$

466 Thus,

$$S_1 = S_2 = b(p_i) P(p_i, j) = 0.6561 \times 0.9 = 0.59049.$$

467 Treating the two incoming signals as independent gives

$$\begin{aligned} P(\text{signal}) &= 1 - (1 - 0.59049)^2 = \textcolor{red}{0.83227}\textcolor{blue}{0.83230}, \\ b(j) &= 0.9 \times \textcolor{red}{0.83227}\textcolor{blue}{0.83230} = \textcolor{red}{0.749043}\textcolor{blue}{0.749071}, \end{aligned}$$

468 which overestimates the correct value.

469 *3.3.3. Solution by Diamond Conditioning (Correct and Efficient)*

470 Conditioning on the fork state, we have

$$b(f) = 0.81.$$

471 If the fork is active, each downstream path has conditional transmission
472 probability

$$P(f, p) R(p) P(p, j) = 0.9 \times 0.9 \times 0.9 = 0.729.$$

473 Therefore,

$$\begin{aligned} P(\text{signal} \mid F = 1) &= 1 - (1 - 0.729)^2 = 0.926559, \\ P(\text{signal} \mid F = 0) &= 0. \end{aligned}$$

474 Using (12),

$$\begin{aligned} P(\text{signal}) &= P(\text{signal} \mid F = 1)P(F = 1) + P(\text{signal} \mid F = 0)P(F = 0) \\ &= 0.926559 \times 0.81 + 0 \times 0.19 \\ &= 0.75051279. \end{aligned}$$

475 Hence,

$$b(j) = 0.9 \times 0.75051279 = 0.675461511,$$

476 which matches the exact path-enumeration result.

477 This example shows the central idea behind IPA: the dependence induced
478 by a reconvergent structure can be handled exactly by conditioning only on
479 the shared upstream variables that generate the dependence.

480 3.4. IPA Workflow and ~~algorithm~~Algorithm

481 The Information Propagation Algorithm computes node reliability in a
482 single forward pass over the DAG. For each node, the algorithm:

- 483 • computes incoming signal probabilities from its parent nodes;
- 484 • applies the closed-form independent update when the parent signals
485 are independent;
- 486 • detects reconvergent structures when dependencies are present;
- 487 • resolves those dependencies through conditional enumeration on shared
488 upstream nodes.

489 This formulation localises exact dependency handling to the subgraphs
490 that require it, ~~while retaining~~. Elsewhere, it retains the simple closed-form
491 updates ~~elsewhere~~.

492 Algorithm 1 summarises the overall ~~IPA procedure~~computation.

Algorithm 1 Information Propagation Algorithm (IPA)

```

1: Compute a topological ordering of  $G$ 
2: for each node  $v \in V$  in topological order do
3:   if  $v \in S$  then
4:      $b(v) \leftarrow R(v)$ 
5:   else
6:     Compute  $S_i = b(p_i) P(p_i, v)$  for all  $p_i \in \text{Pa}(v)$ 
7:     if incoming parent signals are independent then
8:        $b(v) \leftarrow R(v) [1 - \prod_i (1 - S_i)]$ 
9:     else
10:      Identify reconvergent (diamond) structures affecting  $v$ 
11:      Resolve dependencies by conditional enumeration
12:      Compute  $b(v)$  from the conditioned signal probabilities
13:    end if
14:  end if
15: end for
16: return  $\{b(v)\}_{v \in V}$ 

```

3.4.1. Computational Characteristics

For nodes with independent parent signals, the update cost is $O(|\text{Pa}(v)|)$, so the total computational cost is linear in the number of edges for tree-structured networks.

For nodes involved in reconvergent structures, additional cost arises from conditional enumeration over shared upstream components. If a diamond requires conditioning on c shared upstream variables, the local cost scales as $O(2^c)$ rather than with independent of the total number of full path combinations. The practical efficiency of IPA therefore depends primarily on the number and nesting depth of such dependency-inducing structures.

The next section introduces a progressive decomposition strategy that reduces repeated conditioning by replacing resolved diamond subgraphs with statistically equivalent supernodes.

4. Progressive Decomposition of Diamond Structures

~~Progressive decomposition is the mechanism that makes~~ Progressive decomposition aims to make IPA practical on ~~many~~ non-trivial DAGs with reconvergent structure. ~~It preserves exactness by~~ By replacing resolved diamonds with conditionally equivalent supernodes, ~~and it improves efficiency by preventing it~~ preserves exactness and improves efficiency: the same dependencies ~~from being~~ are never conditioned on repeatedly.

~~While conditional decomposition resolves dependencies locally, real~~ Real process networks rarely contain only isolated diamonds. ~~In practical directed acyclic graphs, reconvergent~~ Reconvergent structures often overlap or nest within one another, so that the output of one diamond becomes part of a larger downstream reconvergent structure. ~~In such cases, repeatedly~~ Repeatedly conditioning on the same upstream variables ~~can lead to severe computational redundancy.~~

~~IPA addresses this issue through progressive diamond decomposition. Once a diamond subgraph has been solved, it is replaced by a statistically equivalent supernode that preserves its exact conditional behaviour while simplifying the remaining graph. This reduces repeated conditioning over already-resolved substructures and allows nested reconvergent patterns to be handled incrementally.~~

~~Consider a nested reconvergent network in which an inner diamond appears as part of a path in a larger outer diamond. Without decomposition, the inner structure would need to be re-evaluated each time the outer diamond~~

529 is conditioned on a new upstream state. If the outer and inner diamonds
 530 contain multiple conditioning variables, the resulting enumeration cost grows
 531 multiplicatively.

532 ~~More generally, here causes computational redundancy:~~ for a network
 533 with L nested dependency levels, a direct conditional evaluation may require

$$b(j) = R(j) \cdot \sum_{s_1 \in \{0,1\}^{m_1}} \sum_{s_2 \in \{0,1\}^{m_2}} \cdots \sum_{s_L \in \{0,1\}^{m_L}} P(s_1) P(s_2 \mid s_1) \cdots P(s_L \mid s_1, \dots, s_{L-1}) \phi(s_1, \dots, s_L), \quad (17)$$

534 where m_ℓ is the number of conditioning nodes introduced at nesting level ℓ ,
 535 and $\phi(\cdot)$ is the resulting signal probability at the join node. ~~This cost~~ The
 536 cost of this direct evaluation grows exponentially with both the number of
 537 conditioning variables and the depth of nesting.

538 ~~The purpose of progressive decomposition is to avoid this~~ Progressive
 539 decomposition avoids the repeated re-evaluation ~~by solving inner diamonds~~
 540 behind (17) by solving each diamond once, storing ~~their~~ its exact conditional
 541 behaviour in a supernode, and reusing that ~~information~~ behaviour whenever
 542 the reduced network is evaluated further downstream.

543 4.1. Supernode Representation and Exactness Guarantees

544 Let D be a diamond subgraph with fork-side conditioning set

$$\mathcal{C}_D = \{c_1, \dots, c_m\}$$

545 and join node j_D . Solving the diamond yields the exact conditional signal
 546 probability at the join node for every configuration of the conditioning vari-
 547 ables:

$$\psi_D(s) = P(J_D \mid s), \quad s \in \{0,1\}^m, \quad (18)$$

548 where J_D denotes the event that a signal reaches the join node of the diamond
 549 and s is a binary state assignment to $\mathcal{C}_D$.

550 The resolved diamond is then replaced by a supernode that stores the
 551 map

$$s \mapsto \psi_D(s).$$

552 ~~This supernode acts as a statistically equivalent component in the reduced~~
 553 ~~graph~~ $s \mapsto \psi_D(s)$: whenever the conditioning state s is encountered in a larger

554 downstream computation, the contribution of the original diamond is recovered
 555 exactly through (18), without re-solving the full subgraph. The following
 556 four results make this exact, in order: that a resolved diamond's conditional
 557 map does not depend on the outer context it sits in; that conditioning
 558 on a shared fork is what makes the map well defined; that independent
 559 reconvergent structures at the same join can each be resolved on its own
 560 separating set, without joint conditioning over the combined ancestry; and
 561 that the supernode replacement is therefore lossless downstream.

562 ~~Let D_{in}~~

563 **Lemma 4.1** (Conditional Invariance). Let D be a diamond subgraph ~~embedded~~
 564 ~~within a larger DAG with join j_D and conditioning set $C \subseteq \mathcal{C}_D$, and let $\mathcal{C}_{\text{in}}$~~
 565 ~~be the conditioning variables required to resolve all dependencies internal to~~
 566 ~~D_{in} . Then the conditional signal probability~~

$$\psi_{D_{\text{in}}}(s) = P(J_{\text{in}} | s), \text{ --- } s \in \{0, 1\}^{|\mathcal{C}_{\text{in}}|},$$

567 ~~depends only on the internal structure of D_{in} and the state assignment s , and~~
 568 ~~is invariant to any outer conditioning context that does not alter the internal~~
 569 ~~topology or component probabilities of D_{in} .~~ $\psi_D(s) = P(\mathcal{R}_{j_D} = 1 | \mathcal{R}_C = s), s \in \{0, 1\}^{|C|}$,
 570 be its conditional signal map. If an enclosing computation conditions on
 571 further variables A disjoint from the nodes and edges internal to D , then
 572 $\psi_D(s)$ is unchanged by that outer conditioning.

573 *Proof.* ~~By construction, all statistical dependencies internal to D_{in} are removed~~
 574 ~~once~~ Fix s . Conditional on $\mathcal{R}_C = s$, the event $\{\mathcal{R}_{j_D} = 1\}$ is a deterministic
 575 function, by (2), of the node and edge indicators internal to D . By Assumption 2.2
 576 these indicators are independent of every indicator outside D , in particular of
 577 the variables in $\mathcal{C}_{\text{in}}$ are fixed. For any fixed state A . Hence $P(\mathcal{R}_{j_D} = 1 | \mathcal{R}_C = s, \mathcal{R}_A = a) = \psi_D(s)$
 578 for every outer state a : the outer conditioning changes only the probability
 579 with which a state s , the remaining signal probability at the join node is
 580 determined solely by the nodes, edges, occurs, never the value $\psi_D(s)$ itself.
 581 □

582 The disjointness required by Lemma 4.1 is what identification enforces
 583 (Section 4.2.1): whenever an outer state does fix a variable internal to a
 584 previously resolved diamond, the sub-problem it defines is a different one
 585 and is resolved, and stored, separately.

586 **Lemma 4.2** (Separator Sufficiency). *Let v be a join node and let $C \subseteq F \cap \text{anc}(v)$
 587 be a set of forks such that $\text{infl}(u; C) \cap \text{infl}(u'; C) = \emptyset$ for every pair of distinct
 588 parents $u, u' \in \text{Pa}(v)$, where infl is defined in (3). Then, conditional on $\mathcal{R}_C$,
 589 the parent reachability events $\{\mathcal{R}_u\}_{u \in \text{Pa}(v)}$ are mutually independent, and ~~path~~
 590 ~~structure contained within D_{in} . Any outer conditioning context may affect the~~
 591 ~~probability with which a state s occurs, but it does not change the value of~~
 592 ~~$P(J_{\text{in}} | s)$ itself, provided the internal structure and local probabilities of D_{in}~~
 593 ~~remain unchanged.~~*

$$\underline{b(v | \mathcal{R}_C = s) = R(v) \left(1 - \prod_{u \in \text{Pa}(v)} (1 - P(\mathcal{R}_u = 1 | \mathcal{R}_C = s) P(u, v)) \right)} \quad (19)$$

594 *Proof.* With $\mathcal{R}_C$ fixed, each $\mathcal{R}_u$ is, by (2), a function of the indicators in
 595 $\text{infl}(u; C)$ alone. These sets are disjoint across parents by hypothesis, and
 596 functions of disjoint sets of mutually independent indicators (Assumption 2.2)
 597 are themselves independent. The stated update is then (9) applied conditionally
 598 on $\mathcal{R}_C = s$. $\square$

599 ~~This immediately justifies replacement by a supernode.~~ Lemma 4.2 is
 600 what the unresolved-influence test of (3) is for: identification returns, for
 601 each reconvergent structure, a conditioning set with this separating property,
 602 so the resulting cost is $2^{|C|}$.

603 **Lemma 4.3** (Independent-Substructure Factorisation). *Fix a conditioning
 604 context E and write $u \sim u'$ for parents u, u' of a join v whenever $\text{infl}(u; E) \cap \text{infl}(u'; E) \neq \emptyset$.
 605 Let $G_1, \dots, G_m$ be the equivalence classes of $\text{Pa}(v)$ under the transitive closure
 606 of $\sim$. The events $\{v \text{ is reached through some parent in } G_k\}$, $k = 1, \dots, m$,
 607 are mutually independent, and*

$$\underline{b(v) = R(v) \left(1 - \prod_{k=1}^m (1 - P(v \text{ reached via } G_k)) \right)}, \quad (20)$$

608 *so each group G_k can be resolved on its own separating set C_k (Lemma 4.2)*
 609 *independently of the others.*

610 *Proof.* Distinct classes have disjoint unresolved influence by construction,
 611 so the class events are functions of disjoint sets of mutually independent

612 indicators, hence independent by Assumption 2.2; the product form is (9)
 613 applied to the m group events. $\square$

614 At a join whose parents form m independent groups with conditioning
 615 sets $C_1, \dots, C_m$, joint conditioning over the combined shared ancestry would
 616 enumerate $2^{|C_1| + \dots + |C_m|}$ states. Lemma 4.3 enumerates $\sum_{k=1}^m 2^{|C_k|}$ states instead.
 617 For k parents each depending on its own independent fork, this reduces the
 618 cost from 2^k to $2k$, from exponential to linear in the fan-in.

619 **Lemma 4.4** (Supernode Equivalence). *Let D be a resolved diamond, or
 620 a resolved independent group G_k of Lemma 4.3, with conditional signal map
 621 $\psi_D(s)$. Replacing D by a supernode storing $\psi_D(s)$ preserves the exact source-
 622 to-node reliability of every downstream node in the reduced graph.*

623 *Proof.* ~~For any downstream computation, the~~ The contribution of D to any
 624 downstream computation enters only through the probability that its join
 625 node receives a signal under a given conditioning state. By Lemma 4.1, this
 626 contribution is exactly 4.1 the stored map equals $\psi_D(s)$ for each state s .
 627 Replacing the original diamond by a supernode that returns the same value
 628 $\psi_D(s)$ under the same conditioning state therefore leaves all downstream
 629 reliability calculations unchanged in every outer context that fixes no variable
 630 internal to D , and by Lemmas 4.2 and 4.3 the downstream update combines
 631 this map with the remaining parent signals through the same independent-union
 632 rule used elsewhere in IPA. Both are preserved by the replacement, so every
 633 downstream belief is unchanged. $\square$

634 **Corollary 4.5** (Acyclic Reduction). *Replacing a resolved diamond D , or a
 635 resolved independent group G_k , by its supernode preserves the acyclicity of the
 636 reduced graph, and this holds under repeated replacement across overlapping
 637 and nested diamonds.*

638 *Proof.* The supernode has exactly one outgoing edge, to D 's join, and its
 639 incoming edges are exactly D 's external incoming edges (edges entering D
 640 from outside D), retained with their original source node and direction; no
 641 edge internal to D survives the replacement, and no edge is introduced.
 642 The reduced graph is therefore obtained from G by contracting an induced
 643 subgraph to a single node along its edges, without reversing or adding any
 644 edge. A directed cycle in the reduced graph would need to revisit the
 645 supernode, which is only possible by leaving it through its single outgoing
 646 edge and returning through one of its incoming edges; tracing that path back

through the corresponding nodes and edges of D and the external nodes it passes through yields a directed cycle in G , contradicting the acyclicity of G (Assumption 1). This is not order-dependent: whenever a diamond D' resolved later contains an already-placed supernode among its own parents or internal nodes, D' 's own incoming and outgoing edges are subject to the same argument, so the property is preserved under repeated replacement across nested and overlapping diamonds. $\square$

4.2. Progressive Decomposition Procedure

IPA applies diamond decomposition progressively ~~during topological evaluation.~~ When the algorithm as it processes the network in topological order. When it reaches a join node ~~involved in a reconvergent structure, it :~~ identifies the associated diamond subgraph, determines the conditioning variables required to resolve its internal dependencies, computes the ~~whose~~ parent signals are dependent, it partitions the parents into independent groups by testing pairwise intersection of their unresolved influence, (3) (Lemma 4.3). For each group it finds the earliest fork common to its members and the subgraph this induces, computes the group's conditional signal map ~~of the join node, replaces the entire resolved diamond by conditioning on that fork (Lemma 4.2), and replaces the resolved group by a supernode in the reduced graph (Lemma 4.4).~~ Independent groups then combine by the factorised union rule; a single correlated group with no independent partition reduces exactly to the non-factorised case of Section 4.1.

~~This procedure transforms the original DAG into a progressively simpler equivalent DAG.~~ Unlike the single-fork diamond of Section 3.3, a diamond resolved this way can itself sit inside a larger diamond, so that the graph must be simplified from the inside out: inner diamonds are resolved first, ~~so that larger outer diamonds can~~ and outer diamonds then treat them as supernodes ~~rather than re-evaluating and do not re-evaluate~~ their internal paths.

Figure 5 ~~illustrates the decomposition of a nested reconvergent network shows this on a network with four such diamonds, some nested inside others.~~

~~The topological order ensures that the inner~~

4.2.1. Tracing the example of Figure 5

The network of Figure 5 has three sources and sixteen further nodes; every non-source node has intrinsic reliability 0.9 and every edge has transmission probability 0.9, as in Section 3.3. Node 6 has three parents, 3, 5, and 13,

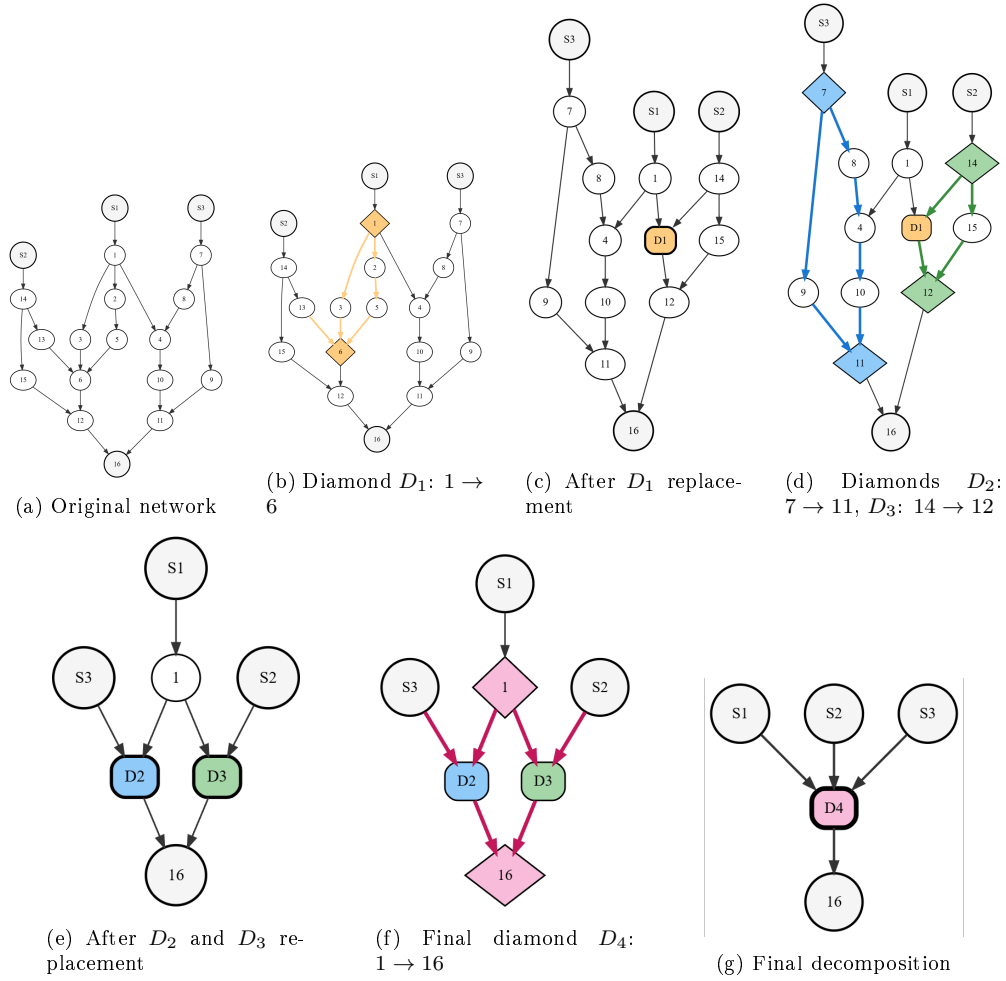


Figure 5: Progressive diamond decomposition in a nested reconvergent network.

but only 3 and 5 trace back to the shared fork 1; the third, 13, traces back through node 14 to a different source and carries no dependency on the other two. IPA therefore does not condition node 6 on a joint state of three variables: it resolves $\{3, 5\}$ as diamond D_1 is detected and resolved before downstream join nodes are processed. Once replaced by a supernode, the reduced graph contains fewer explicit dependency-inducing paths. The same procedure is then applied to (fork 1), then combines the result with node 13's independent signal by the ordinary union rule of Section 3 (Lemma 4.3). Node 4, by contrast, is a join with no diamond at all: its parents 1 and 8 share no common ancestor, so IPA applies the closed-form independent update directly (Section 3) and never conditions on anything there.

D_1 is resolved first, since node 6 is reached before nodes 11, 12, and 16 in topological order. Fork 1 has $b(1) = 0.81$; conditional on 1 being reachable, the union of paths through 3 and 5 has probability $\psi_{D_1}(1) = 0.8890228$, and conditional on 1 being unreachable no signal can reach 6 through either path, so $\psi_{D_1}(0) = 0$. Combining this with node 13's independent signal gives $b(6) = 0.7968435$. Diamonds D_2 (fork 7, join 11) and D_3 (fork 14, join 12) do not depend on D_1 or on each other and are resolved the same way, each conditioning on its own fork. The reduced graph then has a single outer diamond, after which the final outer diamond D_4 can be evaluated using the: fork 1 again, but now covering join 16 through the two already-resolved supernodes at 11 and 12. Fork 1 is conditioned on a second time here, in a different context from D_1 , and the two resolutions are kept as distinct stored sub-problems (Section 4.2.1) because they cover different edges. Table 1 lists all four resolutions; the final belief is $b(16) = 0.8061491$, and each step was checked independently by exhaustive path enumeration of its own sub-network. In this way, dependencies are handled locally and only once.

4.3. Computational Effect of Decomposition and complexity comparison

The benefit of decomposition is not that it removes the exponential nature of exact inference in all cases, but that it prevents repeated enumeration over the same resolved substructures. Without decomposition, nested diamonds may require repeated multi-level conditioning as in (17). With decomposition,

4.2.1. Identification of stored sub-problems

Table 1: Trace of the four diamond resolutions in Figure 5. $w = P(\mathcal{R}_f=1)$; $\psi(1), \psi(0)$ are the conditional signal probabilities at the join given the fork reachable or unreachable, before the join’s own reliability is applied.

Diamond	Join	Fork	Group	w	$\psi(1), \psi(0)$	$b(\text{join})$
D_1	6	1	$\{3,5\} + 13$ (indep.)	0.81	0.8890228, 0	0.7968435
D_2	11	7	$\{10,9\}$	0.81	0.8937451, 0.4782969	0.7333289
D_3	12	14	$\{6,15\}$	0.81	0.9318600, 0.5832879	0.7790682
D_4	16	1	$\{12,11\}$	0.81	0.9139431, 0.8180383	0.8061491

Algorithm 2 gives the identification procedure itself, run at each join whose parent signals are dependent. It partitions the join’s parents into independent ancestry-groups (Lemma 4.3), finds each group’s covering fork, and recurses into nested diamonds under an extended context — this is the recursive partition-and-recurse step that Algorithm 1’s propagation loop invokes in a single line (“identify reconvergent structures affecting v ”). A node is excluded from a conditioning set only when it is provably degenerate: fixed at prior exactly 0, or fixed at exactly 1 *and* a graph source. Excluding any other prior-1 node is unsound here — it would silently drop a real diamond whenever every prior in the network happens to equal 1, which is exactly the input convention of the benchmark grid network of Section 5.1.

A resolved diamond is identified for reuse by its edge list together with the conditioning context it was resolved under, not by its edge list alone. The same edge list can arise from a fork that is free or already fixed by an outer context E , because cutting the incoming edges of an already-fixed node changes nothing in the induced subgraph; the two occurrences are nonetheless different sub-problems, since Lemma 4.1 requires the outer context to fix no variable internal to the diamond. IPA keys each stored diamond D , with edge list $\mathcal{E}_D$ and covering fork f , on

$$\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D))), \quad (21)$$

the edge list together with the new fork and whichever already-fixed forks from the outer context appear among the diamond’s own endpoints $V(\mathcal{E}_D)$. Two occurrences of the same edge list under different outer contexts receive different keys and are resolved, and cached, separately; identical sub-problems are resolved once. A conditioning node already fixed by an outer context contributes only one term to the state enumeration of Section 4.4, not two.

so nesting further levels of context does not double the enumeration already performed at each level.

4.3. *Complexity and Relation to Established Exact Methods*

IPA's cost admits an exact per-instance expression. Let the recursion of Section 4.2 resolve diamonds $d = 1, \dots, M$, counting nested resolutions, with conditioning sets C_d and internal edge sets E_d . The work performed is

$$W = \sum_{d=1}^M 2^{|C_d|} \cdot O(|E_d|), \quad (22)$$

since each diamond is resolved by enumerating the $2^{|C_d|}$ states of its conditioning set, and each state requires a propagation over the diamond's internal structure. Every term of (22) is available before enumeration begins, because identification produces the conditioning sets before any state is enumerated; W is a sound a-priori upper bound on the cost of an analysis, computable in advance for a given network. This bound is not the realised cost: memoisation under (21) reduces the work performed to the number of distinct conditional sub-problems encountered. The factorisation of Lemma 4.3 reduces the exponents themselves in favourable cases: at a fan-in of k independent branches, from 2^k states to a number linear in k , without changing the worst-case class.

~~At a high level, if a nested network contains diamonds $D_1, \dots, D_M$ with conditioning-set sizes $e_1, \dots, e_M$, then progressive decomposition avoids repeated multiplicative re-enumeration and replaces it with a~~

Relation to cutset conditioning and junction trees. Section 1.1 positions IPA as a specialisation of cutset conditioning [23]. Classical cutset conditioning selects a single global cutset whose state enumeration renders the whole network singly connected, at cost exponential in the cutset size. IPA identifies, at each reconvergence it visits, a local separating set of the parents' shared ancestry (Lemma 4.2), conditions on it, and recurses, so the realised cost is a sum of local ~~conditional evaluations of order~~

$$O\left(\sum_{i=1}^M 2^{e_i}\right),$$

~~provided that previously resolved diamonds can be reused through their stored conditional maps.~~ exponentials, not one global exponential. Tree-structured

regions of the network are traversed by the closed-form updates of Section 3 at linear cost, and Lemma 4.3 prevents joint conditioning over forks that are independent given the current context. The maximum conditioning-set size on a network is bounded by the size of the largest set of mutually entangled shared ancestors, itself bounded by the treewidth of the network. IPA sits in the same parametric complexity class as the junction-tree algorithm [26] and a well-ordered binary decision diagram: all are exact and fixed-parameter tractable in the relevant width, and none escapes the #P-hardness of the underlying problem. Section 5.2 reports a measured comparison of conditioning-set size against decision-diagram size across the validation corpus.

~~This should be interpreted as an *effective* complexity reduction relative to repeated nested conditioning, not as a universal-~~

Adversarial structure. Two constructed families probe the two ends of this cost. The family fanin- k consists of k structurally independent forks that each reconverge, on their own branch, at a single shared join; since no two forks share ancestry, Lemma 4.3 conditions each one separately, and identification resolves exactly $2k+1$ sub-problems for every k tested. The family mesh- w is the opposite extreme: a width- w reconvergent lattice with no independent partition, a single fully correlated block on which factorisation does not apply, so the conditioning-set size and the realised work both grow with w . Table 2 reports both families against an independent decision-diagram method built under dynamic variable reordering: independent structure is resolved at a cost linear in the fan-in, while correlated structure pays the full width-exponential cost of (22). A real published benchmark circuit family, assigned the same worst-case bound for all graph topologies. ~~In highly entangled reconvergent networks with many contextual entry points, reuse may be limited and the practical cost may still grow rapidly. input convention, is examined in Section 5.5.~~

~~Table ?? summarises the qualitative effect of progressive decomposition on different update types.~~

~~[Table removed: “Qualitative complexity comparison for node updates with and without progressive decomposition” — Update Type / Without Decomposition / With Decomposition, rows: Single parent update ($O(1)/O(1)$); Independent multi-parent update ($O(|Pa(v)|)/O(|Pa(v)|)$); Single diamond with c conditioning nodes ($O(2^c \cdot d)/O(2^c)$); Nested diamonds (repeated nested conditioning/accumulated local conditioning). Footnote: “Here d denotes the size of the diamond subgraph, and c denotes the number of condi-~~

809 tioning variables required to resolve its internal dependencies.”]

Table 2: Complexity on the two adversarial families: realised sub-problems for IPA (with factorisation) against a sifted reduced-ordered binary decision diagram, in operation-count and wall-clock terms.

Family	param	IPA sub-problems	diagram nodes	IPA time	diagram time
fan-in- k	8	17	474	<1 ms	~1–2 s
fan-in- k	16	33	802	<1 ms	~1–2 s
mesh- w	4	20,603	700	0.36 s	1.19 s
mesh- w	5	170,001	805	3.9 s	1.2 s
mesh- w	8	3,895,252	2,621	159.5 s	2.9 s
mesh- w	9	5,426,417	3,335	5925 s	3.8 s

810 ~~The remaining computational bottleneck of~~ At $w=9$ the sub-problem
811 ~~count grows by only $1.39\times$ over $w=8$ (3,895,252 to 5,426,417) while IPA’s~~
812 ~~wall-clock time grows by $37\times$ (159.5 s to 5925 s); $w=10$ exceeded a 180-second-per-step~~
813 ~~time budget during identification and was not completed. The sub-problem~~
814 ~~count of (22) tracks realised work only up to a point: at the largest conditioning-set~~
815 ~~sizes reached in this family, memory pressure within individual sub-problems~~
816 ~~begins to dominate wall-clock cost before it is reflected in the sub-problem~~
817 ~~count itself, and the approach is governed not simply by network size, but~~
818 ~~by the number, size, and nesting depth of dependency-inducing diamond~~
819 ~~structures as explored empirically in the case studies that follow~~
820 ~~decision diagram’s wall-clock cost — already lower at every mesh- w point tested~~
821 ~~— pulls further ahead than the sub-problem-count comparison alone would~~
822 ~~suggest.~~

823 *Applicability beyond source-to-node reachability.* Cutset conditioning already
824 applies to general Bayesian-network inference: IPA specialises that method to
825 source-to-node reachability, and this specialisation does not generalise back
826 to arbitrary probabilistic graphical models. Its closed-form update rules,
827 the independent-union combination of parent signals and the context-keyed
828 supernode cache, are derived for the monotone reachability structure of
829 (2) and do not carry over directly to a Bayesian network with arbitrary
830 conditional probability tables. What transfers is the width-governed complexity
831 class established above. This distinction also bears on the imprecise propagation
832 of Section 2.5: for classical, precise Bayesian networks, bounded treewidth
833 guarantees polynomial exact inference, but this guarantee does not survive

the move to imprecise, credal parameters, where bounded-treewidth inference remains NP-hard in general [33], and the only known tractable relaxation is an approximation scheme; no known exact one exists for that case [34]. IPA’s width-governed *exact* imprecise result (Section 5.3) is specific to the monotone arithmetic structure of the reachability problem and the explicit diamond-conditioning construction; it does not imply that general credal-network inference could be made exact at the same cost.

4.4. *Implementation in Julia* *Numerical Realisation*

IPA is implemented in Julia as ~~both~~ part of the InfoPropFramework.jl package and the standalone library InformationPropagationAnalysis.jl. ~~In both cases, the algorithm’s functions are exposed in ,~~ with diamond identification exposed in DiamondDecompositionModule and belief propagation in ProbabilityPropagationModule. ~~The module validates network input contracts, then propagates node beliefs from source-to-sink in topological order,~~ following Section 3 and Algorithm 1. The propagation flow is type-parametric.

$$T <: \text{Union}\{\text{Float64}, \text{Interval}, \text{pbox}\},$$

~~One core engine is with one core engine~~ shared across all three ~~probability representations , with representations and~~ type-specific ~~arithmetic~~ dispatch through common ~~arithmetic~~ helpers (addition, multiplication, complement, subtraction, and product/sum operations).

~~For multi-parent updates ,~~ Multi-parent updates are evaluated in the product form $1 - \prod_i (1 - p_i)$. This is equivalent under independence to the ~~alternating inclusion-exclusion is evaluated in product form,~~

$$1 - \prod_i (1 - p_i),$$

~~which is equivalent to the alternating-sum form under independence but is numerically more stable , especially sum,~~ but the product form is numerically stable and avoids the spurious widening the alternating sum introduces for interval-valued inputs.

~~At diamond join nodes~~ At a resolved diamond, beliefs are computed by enumerating ~~states of the identified diamond-conditioning set.~~ Each state defines a conditioned subproblem on the corresponding diamonds subgraph, and the join belief is recovered by weighted state aggregation. ~~the states of~~

its *active* conditioning set: a conditioning node whose contextual belief is already pinned to 0 or 1 by an outer conditioning contributes only one term, not two, matching (21)’s treatment of already-fixed forks, and never changes the result, since the dropped state carries zero weight. Two caches support this: one keyed on $\text{hkey}(D)$ (Section 4.2.1) avoids re-identifying the same diamond, and a second, keyed on a diamond’s edge list together with the node priors of a given conditioning state, avoids recomputing a state’s belief when the identical sub-problem recurs under a different diamond or context.

For interval inputs, ~~aggregation uses corner-state evaluation of each conditioning state is evaluated at the corner states of the conditioning bounds, then encloses the result and the result is enclosed in an interval envelope.—;~~ Section 5.3 shows this yields the exact range, by the monotonicity of (2). For probability-box inputs, recombining a diamond’s two conditional branches is a convex combination of two dependent random quantities, not an independent sum: a flat weighted sum of the branch distributions, exact for the point-valued and interval cases, is unsound for probability boxes, since it double-counts the shared dependence on the conditioning weight and can return a bound outside $[0, 1]$. IPA combines the two branches through the operator of Williamson and Downs [28], nesting one such combination per conditioning node so that m nested two-way combinations reproduce the 2^m -state mixture exactly.

~~Resolved diamonds (supernodes) are cached as conditional state-to-belief maps and reused as drop-in equivalents in downstream updates. The cache key includes both the diamond subgraph structure and This recursive structure is shared across all three input types: the current prior state context inside that subgraph.— Supernode reuse is therefore structure-aware number of conditioning states enumerated per diamond, after the memoisation of Section 4.2.1, is the same realised-work quantity reported in Section 4.3. What differs is the price paid per resolved state. For point-valued and interval inputs this price is $O(1)$; for probability-box inputs, the conditioning weight is represented at a chosen discretisation level, and each combination integrates over that discretisation via the convolution of Williamson and Downs [28], at a cost that increases with the discretisation level. Probability-box cost is therefore governed by the same realised sub-problem count as the exact and state-aware: when the same diamonds subgraph is evaluated under different contextual priors, it is treated as a different cache state. This is most important in nested diamonds, where outer-context beliefs can change the local conditional map even when the internal edge set is unchanged.—~~

903 To keep `pbox` propagation logic aligned with the deterministic and in-
904 terval cases, the module inherits arithmetic helpers from the Julia library
905 `ProbabilityBoundsAnalysis.jl` [35], which provides independent convolution
906 operators to compute `pbox` envelope propagation under path dependencies multiplied
907 by a per-state price set by the discretisation level, and Section 5.3 reports
908 this cost in those terms, not against network size alone.

Algorithm 2 Diamond identification at a join node (recursive, factorised)

Require: join node v , outer conditioning context E (initially $\emptyset$)

Ensure: independent conditioning groups for v , each with its diamond subgraph and conditioning set

```
1: for each parent  $p \in \text{Pa}(v)$  do
2:   compute  $\text{infl}(p; E) = (\{p\} \cup \text{anc}(p)) \setminus E$  {Eq. (3)}
3: end for
4: partition  $\text{Pa}(v)$  into groups  $G_1, \dots, G_m$  by union-find:  $p, p'$  share a group
   iff  $\text{infl}(p; E) \cap \text{infl}(p'; E) \neq \emptyset$  {Lemma 4.3}
5: for each group  $G_k$  do
6:    $f_k \leftarrow$  the effective cost is reduced to the accumulation of
local conditioning costs over the diamonds that must be
resolved topologically-highest fork covering  $\geq 2$  members of  $G_k$ 
(or the case where one member is an ancestor of another)
7:   if no such fork exists then
8:     members of  $G_k$  contribute as ordinary non-diamond parents
(Section 3)
9:   else
10:    relevant nodes  $\leftarrow \{v\} \cup G_k \cup \text{anc}(G_k)$ ; induced edges  $\mathcal{E}_{D_k} \leftarrow$  edges
among relevant nodes not crossing into  $E$ 's already-fixed context
11:    conditioning set  $C_k \leftarrow \{f_k\} \cup (E \cap V(\mathcal{E}_{D_k}))$ ; key
 $\leftarrow \text{hkey}(D_k) = (\mathcal{E}_{D_k}, C_k)$  {Eq. (21)}
12:    if a diamond with this key is already resolved then
13:      reuse its stored conditional map  $\psi_{D_k}$ 
14:    else
15:      recurse into  $D_k$ 's own join nodes with context  $E \cup \{f_k\}$ , then
resolve and store  $\psi_{D_k}$ 
16:    end if
17:  end if
18: end for
19: return groups  $G_1, \dots, G_m$ , each with its conditioning set and conditional
map
```

5. Case Studies

This section evaluates IPA in ~~two complementary settings. The first case study uses a benchmark directed grid network to assess numerical accuracy and computational performance against published results for the directed Probability Propagation Method (dPrPm) [24]. The second applies IPA to a set of directed acyclic graphs derived from Pareto-optimal drone logistics networks in order to examine how reconvergent structure and diamond nesting affect the practical tractability of exact reliability analysis in realistic infrastructure systems~~ five parts: a benchmark validation on a grid network (Section 5.1); a corpus-wide validation and cost comparison against an independent exact decision-diagram method (Section 5.2); the imprecise-propagation capability (Section 5.3); an applied case study of a medical drone logistics network for Scotland (Section 5.4); and a real published benchmark family examined under an adversarial input convention (Section 5.5).

Two methodological points hold throughout the section. Every runtime is measured after a warm-up run of the same computation is executed and discarded, so one-off program-initialisation cost is excluded, and the reported time is the minimum of repeated measurements on otherwise idle hardware. All decision-diagram comparisons use CUDD [36], called from the same process and machine as IPA, with dynamic variable reordering enabled unless stated otherwise. Where component reliabilities are widened into intervals or probability boxes for illustration, only components that are not structurally certain are widened: a component fixed at reliability exactly 0 or 1 by the network’s own structure is left at that value, since degrading a structural certainty into a synthetic uncertainty would change the problem being analysed.

5.1. Benchmark Validation on a Grid Network

5.1.1. Benchmark setup

~~Both the proposed Information Propagation Algorithm (IPA) and Tong and Tien’s directed Probability Propagation Method (dPrPm) [24] compute both compute~~ node reliability in directed acyclic networks by propagating local probability information from upstream to downstream nodes. ~~Both methods exploit~~, exploiting the acyclic structure ~~of the graph to ensure so~~ that upstream dependencies are processed before downstream updates.

The two methods differ in how dependencies are represented ~~and resolved~~. dPrPm propagates both marginal and pairwise probabilities and ~~uses local~~

945 approximations to account for dependent interactions. Specifically, when
 946 multiple predecessors influence a node, the update rules explicitly rely on
 947 approximating a approximates a three-node joint distribution for every new
 948 edge introduced. These rules are fundamental when the incoming messages
 949 are combined in an intermediate structure, where the new link addition
 950 affects nodes that have already received some information, requiring a more
 951 nuanced treatment of shared dependencies. Although effective, this approach
 952 ~~requires storing both marginal and edge that introduces a new dependency;~~
 953 this requires storing pairwise probabilities throughout the network, which
 954 ~~can become becomes~~ memory-intensive in large or dense networks.

955 ~~IPA takes a fundamentally different view of network dependencies by~~
 956 IPA instead identifies reconvergent dependency patterns as diamond struc-
 957 tures and resolves them through conditional decomposition ~~∴ Rather than~~
 958 ~~tracking pairwise relationships, it identifies and decomposes these structures~~
 959 ~~into statistically equivalent supernodes. As a result, IPA avoids maintaining~~
 960 ~~or updating pairwise dependencies at complex intersection points, since the~~
 961 ~~relevant~~ (Section 3). It does not track pairwise relationships: each diamond
 962 is decomposed once into a statistically equivalent supernode, and the relevant
 963 dependency structure is captured inherently in the decomposition once in that
 964 decomposition, not maintained node by node.

965 To provide a direct benchmark, IPA is applied to the same 16-node di-
 966 rected grid network studied in [24], shown in Figure 6. This benchmark ~~is~~
 967 ~~useful because it~~ contains multiple reconvergent structures while remaining
 968 small enough for exact verification by explicit ~~path-based computation~~path
 969 enumeration. Following [24], all source nodes are assigned prior reliability ~~1.0~~
 970 1.0 and all directed edges are assigned transmission probability ~~$R_t = 0.909$~~ .

971 5.1.2. Accuracy results

972 Table 3 ~~compares the node reliabilities obtained using IPA with exact~~
 973 ~~values and with~~ brings together three independent quantities at every node:
 974 the value IPA computes, the value a reduced-ordered binary decision diagram
 975 (ROBDD) computes for the same network, and the lower and upper bounds
 976 ~~reported published~~ for dPrPm in [24]. ~~IPA reproduces the exact values up and~~
 977 the ROBDD agree to floating-point precision ~~∴ By contrast, dPrPm provides~~
 978 ~~tight lower and upper bounds that coincide with the exact values for many~~
 979 ~~nodes in this~~ at every node, with a worst disagreement of 1.1×10^{-16} ; the
 980 column reported as exact is therefore not IPA's own output taken on trust,
 981 but a value two independent exact methods reach separately. Against that

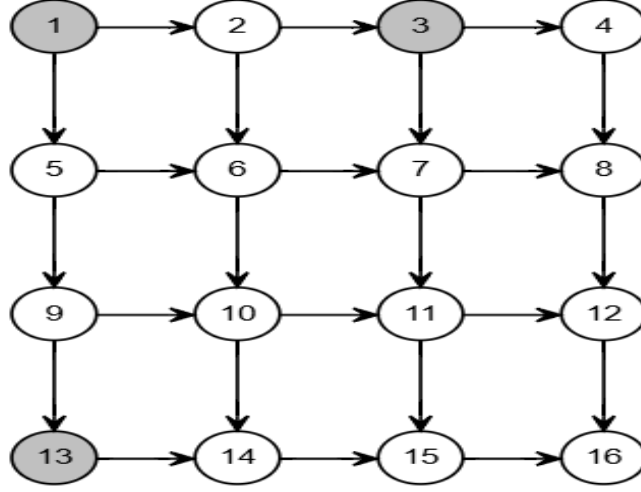


Figure 6: 16-node directed grid network [24]. Source nodes 1, 3, and 13 are coloured in grey.

value, the dPrPm bounds coincide everywhere except node 16, where a gap of 0.00048 remains: the same node the source study itself identifies as the lowest-probability, hardest-to-bound event in the benchmark.

Table 4 compares reports the runtime of IPA with on this benchmark against the runtime reported for dPrPmin [24]. IPA completes the analysis of the benchmark grid network in 2.378ms. Tong and Tien [24] reported a runtime of 0.32s for dPrPm on the same benchmark. Although this is not a controlled implementation-level comparison, it suggests that IPA can provide substantial computational savings while retaining exactness. Only IPA memory usage was profiled in the present implementation; comparable memory data for dPrPm were not available from the source study.

The benchmark study therefore supports two conclusions. First, IPA reproduces the exact node reliabilities for a nontrivial reconvergent DAG. Second, it does so with a very low runtime on this class of network, consistent with the intended use of the method for repeated reliability evaluation in aeyelic process systems. The dPrPm figure is quoted from the source study and was obtained in a different implementation environment, so the comparison is indicative only, and the dPrPm implementation is not publicly available

Table 3: ~~Comparison of node reliability computed for~~ Node reliabilities on the benchmark grid network (~~$R_l = 0.9R_l = 0.9$~~). The exact column is where IPA and an independent ROBDD agree (worst disagreement 1.1×10^{-16}); dPrPm lower/upper bounds and gap are as reported in [24], Table 2; the source prints the node-16 gap as 0.00048, although its printed bounds differ by 0.00052. Source nodes (1, 3, 13) are fixed at reliability 1 and are not reported as dPrPm bounds in the source study.

Node	Exact = IPA	dPrPm lower	dPrPm upper	Gap
1	1.00000000	—	—	—
2	0.99000000	0.99000	0.99000	0.00000
3	1.00000000	—	—	—
4	0.98015430	0.98015	0.98015	0.00000
5	0.90000000	0.90000	0.90000	0.00000
6	0.99510274	0.99510	0.99510	0.00000
7	0.98955925	0.98956	0.98956	0.00000
8	0.89060332	0.89060	0.89060	0.00000
9	0.98100000	0.98100	0.98100	0.00000
10	0.88290000	0.88290	0.88290	0.00000
11	0.97734390	0.97734	0.97734	0.00000
12	0.97456525	0.97457	0.97457	0.00000
13	1.00000000	—	—	—
14	0.97946100	0.97946	0.97946	0.00000
15	0.98497989	0.98498	0.98498	0.00000
16	0.98538853	0.98506	0.98558	0.00048

to reproduce it. IPA and the ROBDD (290 nodes after dynamic reordering; Table 5) are the same "Exact = IPA" computation reported in Table 3, agreeing at every node to 1.1×10^{-16} ; Table 4 reports each method's own build-and-evaluate cost for that already-established agreement. Section 5.2 reports the same IPA-versus-ROBDD cost comparison across every other network in the study, and Section 5.3 carries this network through interval and probability-box propagation.

[Table removed: "Computational performance on the benchmark grid network" — Method / Runtime / Memory, rows: IPA (2.378 ms / 4.94 MB); dPrPm^a (0.32 s / N/A^a). Footnote: ^aResults from [24] — comparable memory profiling data not available.]

The ROBDD comparison was also run beyond the benchmark's own parameterisation, with non-unit node priors and with the boundary case in which every node prior equals one (Section 4.2.1); identification agrees

Table 4: Computational performance on the benchmark grid network. IPA figures are machine-specific (measured on the authors’ current hardware after a discarded warm-up run) and are reported to characterise order of magnitude, not for cross-machine comparison. The dPrPm runtime is quoted from [24] and is indicative only. The ROBDD figure is the time to build the sifted diagram and evaluate every node’s reliability once, measured in its own process after repeated warm-up builds. [†]Memory is the Julia-side allocation only; the diagram’s own storage is held by the CUDD library and is not counted, and its peak during construction was 5,110 live nodes against 290 in the final diagram.

Method	Runtime	Memory
IPA	1.43 ms	1.74 MB
ROBDD (CUDD, sifted)	40 ms	0.08 MB [†]
dPrPm (reported)	0.32 s	n/a

with the ROBDD in both, and the same identification procedure is the one validated across the corpus of Section 5.2.

5.2. ~~Application to Drone Medical Delivery Networks~~ Validation and Cost Comparison Against an Exact Decision-Diagram Method

5.2.1. ~~DAG construction from the multiplex logistics network~~

To evaluate IPA on ~~IPA~~ was compared against an independent exact method, a reduced-ordered binary decision diagram (ROBDD) built with CUDD under dynamic variable reordering, on four groups of networks. The first is a corpus of 129 random and mutated DAGs of 10 to 28 nodes with densities 0.14–0.42 (Table 6). The second is six structured topological families: multi-source, grid/lattice, layered, bridge, series-parallel, and complete. The third is larger random networks of 30 to 50 nodes. The fourth is named structured and real networks (Table 5): the benchmark grid, a ~~realistic infrastructure network~~, the algorithm is applied to drone logistics network designs from Jones et al. [37], who developed a multi-objective optimisation framework for NHS Scotland medical delivery networks. Their work generates Pareto-optimal network configurations by trading off delivery time, capital cost, and network resilience under random node failures ~~power distribution network~~, a gas distribution network (Memphis Light, Gas and Water), a metropolitan transit network (Berlin), and the EPANET Net3 water distribution benchmark.

The original logistics network is represented as a multiplex graph with two transportation layers corresponding to two drone types: **VTOL layer**:

~~short-range-vertical take-off and landing drones with an operational range of 70~~ Every network in these groups was evaluated by both methods. On the corpus, the worst per-node disagreement is 1.1×10^{-16} for point-valued reliabilities, and no interval bound differs by more than 2.8×10^{-16} ; on the named networks of Table 5 it is at most 3.3×10^{-16} . These differences are floating-point rounding; double-precision arithmetic has a unit roundoff of 1.1×10^{-16} , and the two methods sum the same quantities in different orders. Breadth of topology and scale was additionally tested on published Bayesian-network benchmark topologies (bnlearn.com/bnrepository), with every node and link reliability set to 0.9. On 18 of them, of 5 to 1,041 nodes, the Float64 reliabilities and the interval bounds (each reliability widened to $[0.85, 0.95]$) agree with a sifted ROBDD to within 4.4×10^{-16} . Three further networks did not complete under these inputs: IPA propagation on andes (conditioning width about 21) and link, and identification on diabetes, each exhausted memory or a seven-minute budget on a 16 km, capable of serving any feasible location pair within range; **fixed-wing layer:** long-range drones with an operational range of 700 km, restricted to airport-to-airport routes because they require runway infrastructure. Each layer is modelled as a symmetric distance matrix indicating the feasible travel distance between node pairs. GB workstation. Component reliabilities are illustrative, so the test concerns topology and scale, not the systems the benchmarks originally model.

To apply IPA, each Pareto-optimal configuration is transformed into a directed acyclic dependency graph. This DAG is not intended to reproduce the full bidirectional routing behaviour of the original logistics network. Instead, it represents the supply-chain dependency structure through which medical supplies propagate from origin hubs to downstream facilities. Nodes are assigned to four functional levels: Level 0 (Sources): Major hub hospitals that originate supply distribution Level-

Edges are directed from lower levels to higher levels. For nodes within the same level, edges are directed from lower to higher node identifiers in order to preserve connectivity while avoiding cycles. Although the original network matrix encodes bidirectional flight feasibility, the derived DAG representation captures the dependency logic relevant for source-to-node reliability analysis, i.e., downstream nodes depend on upstream nodes for supply access.

The two transportation layers are merged by taking the maximum transmission probability where both drone types can serve the same edge. Edge transmission probabilities are derived from distance using an exponential decay law, so

Table 5: Exactness and complexity of IPA on named structured networks, against the exact ROBDD oracle. Root D. and uniq. D. are the numbers of root and unique diamond structures identified; $|C|_{\max}$ is the largest conditioning set; ROBDD is the node count under dynamic reordering; $|\Delta|_{\max}$ is the worst per-node deviation of IPA from the exact ROBDD reliability.

Network	$ V $	$ E $	
power network	23	27	
grid (benchmark)	16	24	
Karl network	26	74	1 (Re)
counterexample-n15	15	23	
gas network (Memphis)	37	40	2 (Bridges) : Charging stations enabling extended-range
transit network (Berlin)	306	350	
water network (EPANET Net3)	97	119	

that shorter flights have higher success probabilities. Node prior probabilities represent the operational availability of physical facilities and are assigned according to infrastructure type. Major hub hospitals are assigned prior reliability 1.0 as fully reliable supply origins. The comparison also quantifies cost. For each network we report the maximum conditioning-set size $|C|_{\max}$ encountered by IPA, its realised number of distinct conditional sub-problems, and the ROBDD node count under dynamic reordering. As anticipated in Section 4.3, $|C|_{\max}$ tracks the logarithm of the ROBDD size across the corpus (Figure 7), confirming that the two methods are governed by the same structural width. The realised work of the two methods is of the same order, and neither dominates: on some networks IPA resolves fewer sub-problems than the diagram has nodes (42 against 323 on the counterexample network; 142 against 759 on the complete 8-node network), on others more (1961 against 1043 on a layered network). No family in the corpus favours IPA for exact point-valued computation: its value rests on the imprecise-propagation capability of Section 5.3, not on a smaller exact state space.

From a planning perspective, this DAG transformation enables the exact computation of downstream access reliability while also revealing the structural conditions under which exact inference remains computationally feasible across alternative network designs.

Six Pareto-optimal configurations are analysed. These span a wide range of graph sizes and reconvergent dependency structures. Table ?? summarises the basic structural characteristics of

Table 6: Random and mutated DAG corpus (129 graphs), summarised by family. Random DAGs are generated per family at edge probability $p \in \{0.10, 0.15, 0.20\}$ and eight seeds; mutant-n28 is a base $n = 28$ DAG under $+5/-2$ edge mutations. Every graph in every family is exact against the ROBDD oracle ($|\Delta|_{\max} \leq 1.1 \times 10^{-16}$ across the whole corpus). Naive blow-ups are builds exceeding 2.5×10^6 ROBDD nodes under a naive (topological) variable order, all resolved by dynamic reordering.

Family	#graphs	$ V $	$ E $	density	med. ROBDD	naive blow-ups
random-n10	24	10	12–18	0.27–0.40	80	0
random-n12	24	12	15–28	0.23–0.42	197	0
random-n15	24	15	21–35	0.20–0.33	450	0
random-n20	24	20	31–50	0.16–0.26	1164	0
random-n25	24	25	42–73	0.14–0.24	3119	9
mutant-n28	8	28	62	0.16	2562	6
counterexample	1	15	23	0.22	323	0

5.3. Imprecise Reliability Propagation

The results so far establish that IPA matches a well-ordered decision diagram on exact point-valued reliability, at a comparable realised amount of work. This section presents the capability that separates the two: the propagation scheme accepts interval- and p-box-valued component reliabilities directly (Section 2.5).

5.3.1. Interval-valued reliabilities: exact and fast

Because $b(v)$ is non-decreasing in every component reliability (Section 2), its exact range over interval inputs is attained at the two extreme corners of the input box: the lower bound when every component takes its lower reliability, the upper bound when every component takes its upper reliability. This was verified across the full corpus of Section 5.2 by comparing the propagated interval against the decision-diagram evaluation at both corner inputs: the worst discrepancy in either bound is of order 10^{-16} . Interval propagation that ignores reconvergent dependence, by contrast, over-widens the range by up to 0.45 across the corpus, showing that the resulting DAGs-conditioning machinery of Section 4 is as necessary for sound imprecise propagation as it is for exact point values.

The interval computation is also fast. For a one-shot query (given a network, produce the interval reliability), IPA completed faster than the decision-diagram route to the same answer (build the diagram once, evaluate

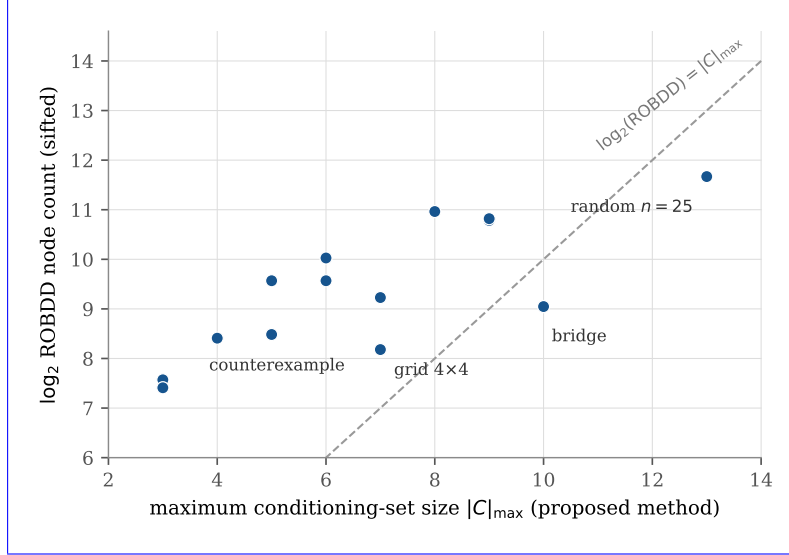


Figure 7: IPA’s maximum conditioning-set size against the logarithm of the sifted ROBDD node count, across the complexity-validation networks. Both quantities measure the same structural width; neither method dominates.

at both corner inputs) on every one of eight corpus families tested, by factors ranging from 3.9 to 99.9 (Table 7). The diagram route’s time is nearly flat, between 44 and 57 ms, across the six smallest networks of the table, and the same holds across the corpus of Table 6: the sifted build takes a median of 38 to 45 ms for the 84 graphs whose diagrams have fewer than 1,000 nodes, and grows only beyond that. These ratios therefore partly reflect a per-build cost that does not depend on network size at this scale, and they are smaller on the larger networks. This claim is scoped to the one-shot cost, in which the diagram’s construction cost counts against it; a workload that amortises a single construction across many repeated queries on a fixed network is a different scenario, not tested here, and could favour the diagram route instead.

To characterise the reconvergent complexity more directly, Table ?? reports the number of identified diamonds, the number of diamond join nodes, the ratio of diamonds to diamond joins (D/DJ),

5.3.2. Probability-box reliabilities: guaranteed distributional bounds

At a join resolved by conditioning with p-box inputs, the belief is a convex combination $W \cdot A + (1 - W) \cdot B$, where W is the uncertain probability of

~~Structural characteristics of Pareto-optimal drone network DAGs Joins = nodes with multiple parents. DiamJoins = join nodes that are diamond join points (reconvergent paths from shared ancestors).~~

Table 7: One-shot interval reliability: IPA against the decision-diagram route (build once, evaluate at both corner inputs), warm-measured. IPA is faster on every family tested; the comparison is scoped to one-shot queries.

<u>Network</u>	<u>V</u>	<u>E</u>	<u>ROBDD</u>	<u>IPA (ms)</u>	<u>ROBDD route (ms)</u>	<u>ratio</u>
<u>counterexample</u>	<u>15</u>	<u>23</u>	<u>323</u>	<u>0.819</u>	<u>44.603</u>	<u>54.5</u>
<u>multi-source</u>	<u>25</u>	<u>57</u>	<u>5622</u>	<u>22.422</u>	<u>175.497</u>	<u>7.8</u>
<u>grid 4×4</u>	<u>16</u>	<u>21</u>	<u>248</u>	<u>0.517</u>	<u>51.654</u>	<u>99.9</u>
<u>layered 4×6</u>	<u>23</u>	<u>35</u>	<u>484</u>	<u>4.619</u>	<u>52.075</u>	<u>11.3</u>
<u>bridge</u>	<u>16</u>	<u>25</u>	<u>529</u>	<u>11.571</u>	<u>48.515</u>	<u>4.2</u>
<u>series-parallel</u>	<u>23</u>	<u>32</u>	<u>215</u>	<u>2.429</u>	<u>57.304</u>	<u>23.6</u>
<u>complete-8</u>	<u>8</u>	<u>28</u>	<u>685</u>	<u>1.537</u>	<u>49.466</u>	<u>32.2</u>
<u>random $n=25$</u>	<u>25</u>	<u>53</u>	<u>4146</u>	<u>27.844</u>	<u>107.875</u>	<u>3.9</u>

the conditioning state and A, B are the uncertain conditional beliefs of the two branches. A and B are functions of shared upstream reliabilities and are therefore dependent; treating the combination as an independent sum is unsound and can assign probability mass outside $[0, 1]$. IPA instead integrates over the distribution of W , combining the branch distributions at each weight level, with the branch dependence bounded by the Fréchet–Hoeffding inequalities [28]. Soundness of the resulting bounds is inherited from that classical result; the computed pair of distribution functions encloses the true distribution of the ~~maximum-observed-nesting-depth~~ node reliability under any dependence between the branches. A tighter variant restricts the dependence bound to non-negative dependence, motivated by both branches being non-decreasing functions of the shared upstream reliabilities and so unable to be negatively dependent; this variant produced no soundness violation in any validation configuration tested (dozens of topology, input-shape, and uncertainty-width combinations, each checked against Monte Carlo ground truth), but a general proof of its soundness under partial dependence information of this kind is not available and is future work. Every headline claim in this paper uses only the provable Fréchet-based bound.

~~The main computation cost of the algorithmic flow comes from the complexity of diamond decomposition and super-node creation and reuse rate. Table ?? compares the diamond structure characteristics between all six networks.~~

Bound tightness, as distinct from soundness, depends on the network's structure. Across a sweep of input-distribution shapes, uncertainty widths, and reconvergence regimes on the benchmark grid network of Section 5.1, the certified band ranged from approximately 0.18 of the unit range under high-reliability, weakly reconvergent conditions to approximately 0.70 under strongly reconvergent, fully uncertain conditions, with zero soundness violations across the sweep (Figure 8). Reconvergence depth combined with input uncertainty drives tightness; the shape of the input distribution is secondary.

~~Diamond-structure complexity metrics for the derived drone network DAGs.~~

~~D/DJ = diamonds per diamond join, indicating the average concentration of nested diamonds around each reconvergent join node. MaxDepth = maximum observed nesting depth of diamond structures.~~

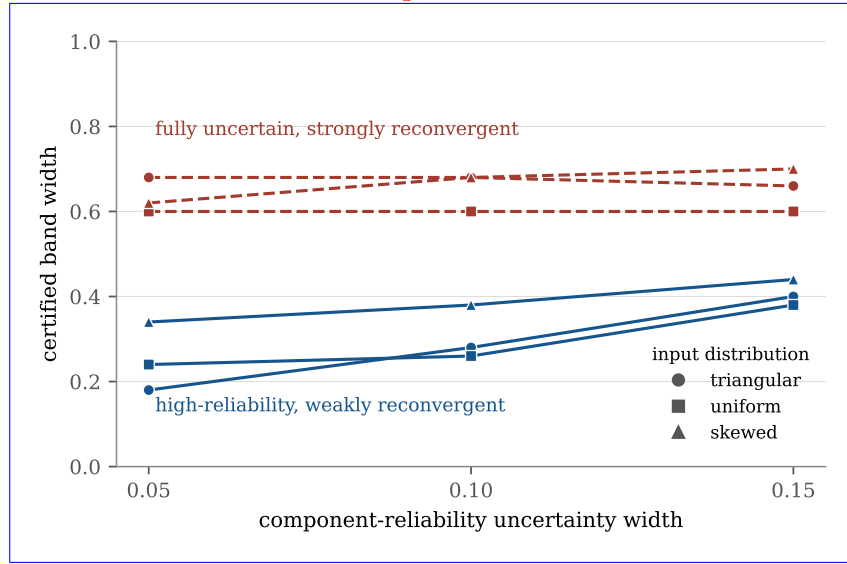


Figure 8: Tightness envelope of the certified p-box bound on the benchmark network: band width against component-reliability uncertainty width, for three input-distribution shapes and two reconvergence regimes. Soundness holds in every configuration; only tightness varies, and it is driven by the regime, not the input shape.

The diamonds-per-diamond-join ratio (D/DJ) indicates how many diamonds are concentrated at each diamond join point on average. A higher D/DJ suggests that individual join nodes participate in more nested diamond structures. PP1 has the highest D/DJ (11.5), consistent with its deep nesting structure. PP2 has the lowest D/DJ (4.2) among networks with diamonds, reflecting

its wide, shallow topology with many diamond joins but less nesting per join. Computational cost is controllable rather than fixed. Interval propagation costs a small constant factor over point-valued propagation (about 1.8 in our measurements: 0.465ms against 0.819ms on the counterexample-n15 network of Table 5), while p-box propagation grows with the discretisation level of the distribution bounds: 1.54s, 8.68s, 51.02s, and 339.02s at 25, 50, 100, and 200 levels respectively on that same counterexample network, each measured as a single propagation in its own fresh process (a shared warm process inflates later, larger measurements, so one clean process per measurement is used throughout this paper). The resulting exponent is approximately 2.6 over this range and increases toward its upper end; propagation beyond 200 levels is impractical on this network and is not used. Bound tightness is therefore always stated at the discretisation level used. Distributional propagation is not cheap, but its cost is controllable and the bounds are rigorous at every setting.

The same realised-work quantity that governs Float64 and interval cost (Section 4.3) also governs p-box cost, at a fixed discretisation level. Figure 9 plots p-box propagation time at steps= 50 against the realised sub-problem count already reported for the exact and interval comparison, across 14 networks spanning a $141\times$ range in that count. The relationship is close to linear on log-log axes, with time per sub-problem clustered within roughly 0.14–0.26s throughout; the raw conditioning-set-width bound $2^{|C|}$, by contrast, does not correlate this cleanly, since two networks with similar bounds can differ substantially in realised sub-problems after memoisation.

The D/DJ-ratio correlates with nesting depth across these configurations: higher D/DJ networks tend to have deeper nesting. However, the relationship is nondeterministic: PP3 and PP5 have similar D/DJ ratios (8.7 and 8.4) and identical maximum depth (16) but different computation times (281s vs 256s), showing that other structural factors also affect performance.

5.3.3. *Effect of diamond structure on runtime*

5.3.3. *A certified decision bound from a single propagation*

The dominant computational cost in IPA arises from the decomposition of reconvergent structures, the construction of supernodes, and the extent to which resolved supernodes can be reused across downstream conditioning contexts. In the absence of progressive decomposition, the cost of evaluating nested diamonds grows exponentially with the number of conditioning variables and increases further with the size of the nested subgraphs.—propagated

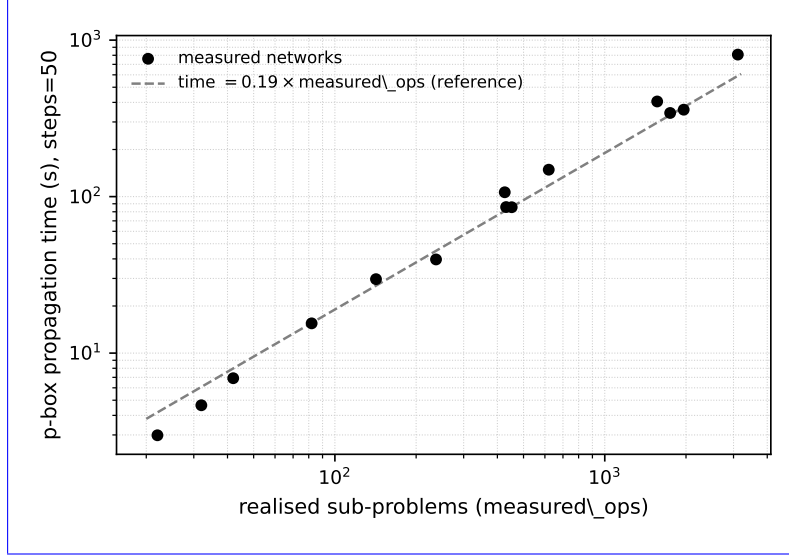


Figure 9: p-box propagation time at a fixed discretisation level (steps= 50) against the realised sub-problem count (measured_ops) already reported for the exact/interval cost comparison (Section 4.3), across 14 networks spanning a $141\times$ range. Each point is a single clean measurement, one propagation per fresh process. The near-linear relationship shows that the same realised-work quantity governing Float64/interval cost also governs p-box cost, at a roughly constant price per resolved sub-problem for a given discretisation level.

distribution bounds certify decision-relevant probabilities directly. For a stated reliability requirement x^* at a given node, the propagated p-box yields guaranteed lower and upper bounds on $P(b(v) \leq x^*)$, the probability that the node fails its requirement, from one propagation. On the benchmark network at a representative requirement $x^* = 0.95$, the certified band on this probability had width between 0.02 and 0.12 across the regimes tested (Table 8). Matching that precision by simulation, planned conservatively without foreknowledge of the answer, would require between 267 and 9,604 samples of the network’s uncertain inputs, and the result would remain a statistical estimate, not a guarantee. A decision-diagram method has no analytic route to this bound and would itself fall back on simulation; this certified-bound capability, not raw speed, is the concrete engineering value of imprecise propagation.

Three structural factors were observed to strongly influence runtime in the six derived networks:—

1. **Diamond nesting depth.** Deeply nested diamonds require repeated layers of conditional evaluation and are the strongest driver of computational growth.
2. **Conditioning-set size.** Diamonds with many shared upstream nodes require more conditioning states.
3. **Contextual entry points.** When a resolved diamond receives additional context-dependent inputs from outer conditioning states, reuse of its supernode representation becomes less effective.

Table ?? summarises the observed relationship between nesting depth and practical applicability given the structure of the drone network.

Empirical tractability spectrum for exact reliability analysis with IPA on the derived drone network DAGs.

Table 8: Certified bound on the requirement-violation probability $P(b(v) \leq 0.95)$ at the benchmark network’s target node, from a single propagation, against the Monte Carlo sample count required to match the certified precision under worst-case a-priori planning.

<u>Regime</u>	<u>Uncertainty width</u>	<u>Certified band</u>	<u>Band width</u>	<u>MC samples</u>
<u>perfect components</u>	<u>0.05</u>	<u>[0.000, 0.020]</u>	<u>0.02</u>	<u>9,604</u>
<u>perfect components</u>	<u>0.10</u>	<u>[0.000, 0.100]</u>	<u>0.10</u>	<u>385</u>
<u>perfect components</u>	<u>0.15</u>	<u>[0.000, 0.120]</u>	<u>0.12</u>	<u>267</u>
<u>uncertain components</u>	<u>0.05–0.15</u>	<u>[0.980, 1.000]</u>	<u>0.02</u>	<u>9,604</u>

This table should be interpreted as an empirical summary of the present dataset rather than as a universal complexity law. Nevertheless, it provides a useful practical guideline: in these networks,

5.4. Applied Case Study: A Medical Drone Logistics Network for Scotland

With the methodology validated in Sections 5.1–5.3, this section applies IPA to a real infrastructure planning problem: the conceptual medical drone delivery network for Scotland designed by Jones et al. [37]. Jones et al. optimised station placement and network structure over real hospital, airport, and candidate-station locations, using stated infrastructure specifications and two drone types: a short-range vertical take-off and landing craft (nominal range 70 km) and a long-range fixed-wing craft (nominal range 700 km). Their optimisation trades off delivery time, capital cost, and resilience, and their framework uses point probabilities. The question addressed here is how confidently each location in a candidate design can be said to be reachable

from the supply hubs, given the uncertainty in the component availability assumptions.

5.4.1. *Building the reliability model from the source study*

The source study’s resilience analysis treats non-hub locations as failing independently with probability 0.2 and hub locations as always available. The reliability model keeps this structure but represents non-hub availability as the interval $[0.75, 0.85]$, centred on the source study’s own 0.8 success probability. The 0.2 figure is an assumption in the source study, not a value derived from data, so the interval turns the analysis into a sensitivity check on that single assumption. Hub locations keep availability 1.

Connections follow the source study’s construction rule: a connection between two locations exists, for a given drone type, exactly when the distance between them is within that type’s nominal range. Where both drone types can serve a connection, the link is available if either succeeds, and the ~~transition from interactive to impractical runtime is relatively sharp once nesting depth exceeds approximately 12–16 levels~~ two transmissions are independent. The model adds one element to connection reliability. The source study holds weather constant while noting that weather-driven range variation is an uncertain quantity that would need expert elicitation. A connection well within a drone’s nominal range is treated as reliable, as in the source study. A connection within the final ten per cent of the nominal range, an illustrative weather-derating margin, is given the vacuous transmission probability $[0, 1]$, which expresses that adverse weather may or may not close the route. All other probabilities come from the source study.

The dependency graph is directed from the hub tier outward. The source study’s connections are undirected, since a route can be flown either way, but the reliability question is whether supply reaches each facility from the hubs. The directed graph follows the study’s hub-and-spoke architecture, and source-to-node reachability is computed on the resulting acyclic structure.

5.4.2. *Pareto Network Results*

The source study’s optimised network layouts are not published, so three configurations are built as proxies for the qualitative character of three trade-off points it describes. They are not reproductions of an optimisation output. The first is a fixed-wing-reliant centralised design that serves remote locations through a hub backbone (217 locations, 263 connections). The second is a densely interconnected short-range design (242 locations, 1,753 connections).

The third is a minimal-investment design that keeps only pre-existing, non-optional infrastructure (230 locations, 1,648 connections). The centralised design is sparse (mean in- and out-degree 1.2, maximum out-degree 27 at a hub) and close to tree-like; the other two are far denser (mean degree 7.2, maximum out-degree 104 at a hub), with individual facilities served by as many as 16–17 alternate incoming routes, the same per-facility redundancy that produces the conditioning widths reported below.

Table ?? compares IPA runtime for each network configuration, ordered by computation time.

5.4.2. *Redundancy and the practical boundary of exact analysis*

IPA runtime across Pareto-optimal drone network configurations.

The results show that runtime is governed primarily by reconvergent nesting depth rather than by graph size alone. For example, PP2 has 171 nodes and 219 diamonds but completes in 0.31 seconds as a result of its shallower nesting (depth 9). Comparing this to PP3 which has fewer nodes (98) and fewer diamonds (165) but still requires 281 seconds because of deep nesting (depth 18). The reconvergent structure of each configuration is controlled by a network-design parameter: the number of alternate routes provisioned per location. This is a redundancy choice in the network design, not a parameter of the analysis method. Connecting every operationally plausible pair of locations makes exact diamond identification itself impractical. Identification demanded more than 25 GB on a 16). On the extreme case, PP1 with nesting depth 18 falls beyond the practical range for interactive exact analysis. Comparing computational costs, it is clear that the transition between tractable and intractable occurs sharply. Networks with nesting depth ≤ 12 complete in under one second regardless of diamond count. The two larger networks with depth 16 GB workstation within about fifteen minutes and did not complete. Limiting the provision to sixteen alternate routes per location brings the conditioning requirement to 16 take a lot more time and by depth 18, the run time starts to exceed practical limits. These results support the central claim of the paper: the principal determinant of tractability in exact DAG reliability analysis is not the raw number of nodes or edges, but the arrangement and nesting depth of dependency-inducing reconvergent structures in both dense configurations. Further increases in provisioned routes changed neither the requirement nor the runtime materially, which indicates that the reconvergent structure of this network saturates at that level. Full exact interval propagation on the densest configurations completes

in under 25 seconds, and on the centralised configuration, whose conditioning requirement is 8, in well under one second. The practical boundary of exact analysis on this network is therefore a measured quantity expressed through a design parameter: more provisioned redundancy improves resilience and raises the cost of verifying it exactly.

The comparison of Section 5.2 was repeated on these networks. On the centralised configuration the two methods agree to floating-point precision, and IPA’s one-shot interval computation takes 0.008s against 0.463s for the decision-diagram route to the same interval, about 56 times faster (diagram size 2,667 nodes). On the minimal-investment topology with six alternate routes per location, the agreement is 1.1×10^{-16} and IPA takes 0.104s against 1.579s, about 15 times faster (diagram size 4,968 nodes). With sixteen alternate routes per location on the same topology, the decision-diagram build did not complete within a practical computational budget under either of two variable-ordering strategies, while IPA completed the full exact computation in under 25 seconds. Both methods are governed by the same structural width and both are practical at low redundancy. On this network, measured along a real redundancy design parameter, IPA remains practical at a redundancy level where the decision-diagram build does not; the crossover between the two has not been located precisely, and the result is specific to this network and not a general statement about decision-diagram methods.

5.4.3. *Summary of the case-study Reliability findings*

The two case studies support three main conclusions. First, IPA reproduces exact node reliabilities on a benchmark reconvergent DAG and compares favourably with the published dPrPm results. Second, IPA remains computationally efficient on realistic infrastructure DAGs when reconvergent structures are shallow or moderately nested. Third, the practical limit of exact analysis is controlled primarily by the depth and reuse characteristics of nested diamond structures rather than by overall network size alone.

These observations motivate the use of IPA as an exact reliability analysis tool for a broad class of acyclic process systems, while also clarifying the structural regimes in which approximation methods may still be required. Table 9 summarises the propagated interval reliabilities. In the centralised configuration the propagated bounds are narrow (mean band width 0.089, maximum 0.100). Each location’s reachability essentially inherits the availability band assumed for the locations themselves, so the network adds little uncertainty, but it also adds little protection: the design is tree-like, and a single-route

failure disconnects service. In the two higher-redundancy configurations the best-connected locations are reachable with near-certainty, while the worst-served locations show a decision-relevant spread. The least certain facility, Islay Hospital, an island site, has reachability probability bounded between 0.56 and 0.72. A point-probability analysis cannot distinguish a facility whose reachability is confidently 0.6 from one whose reachability lies anywhere between 0.56 and 0.72, yet the two call for different interventions. Figure 10 colours the network by the lower reliability bound and by band width. It identifies the facilities where additional redundancy, or better component-availability data, would most improve either the network or our knowledge of it, and these are concentrated in the islands and the far north.

Table 9: The three proxy configurations: structure, conditioning requirement, and the propagated reliability bounds. Every belief is contained in $[0,1]$ with lower $\leq$ upper (soundness); the worst-served facility in both dense configurations is Islay Hospital.

Configuration	$ V $	$ E $	$ C _{\max}$	mean band	max band	worst facility
fixed-wing centralised	217	263	8	0.089	0.100	[0.750, 0.850]
dense decentralised	242	1753	16	0.094	0.160	[0.562, 0.722]
concentrated minimal	230	1648	16	0.093	0.160	[0.562, 0.722]

5.5. ~~Limitations and Future Work~~IPA provides exact source-to-node reliability values for Adversarial Structure

Section 4.3 used two constructed families, fan-in and mesh, to show where IPA’s factorisation helps and where it cannot. This section asks the same question of a real, independently published family, not a hand-built one: the ISCAS85 combinational circuit benchmarks [38], a suite of digital logic circuits represented natively as directed acyclic graphs by resolving reconvergent dependencies through conditional decomposition. Its main limitation is therefore not approximation error, but computational tractability in the presence of deeply nested dependency structures. Every circuit is assigned the same illustrative, non-degenerate component reliability (0.9, uniformly) used throughout this corpus, which is itself a deliberate worst case here: identification excludes a node from a conditioning set only when its reliability is fixed at exactly 0 or 1 (Section 4), so a non-degenerate value everywhere leaves no reconvergent structure excluded from conditioning.

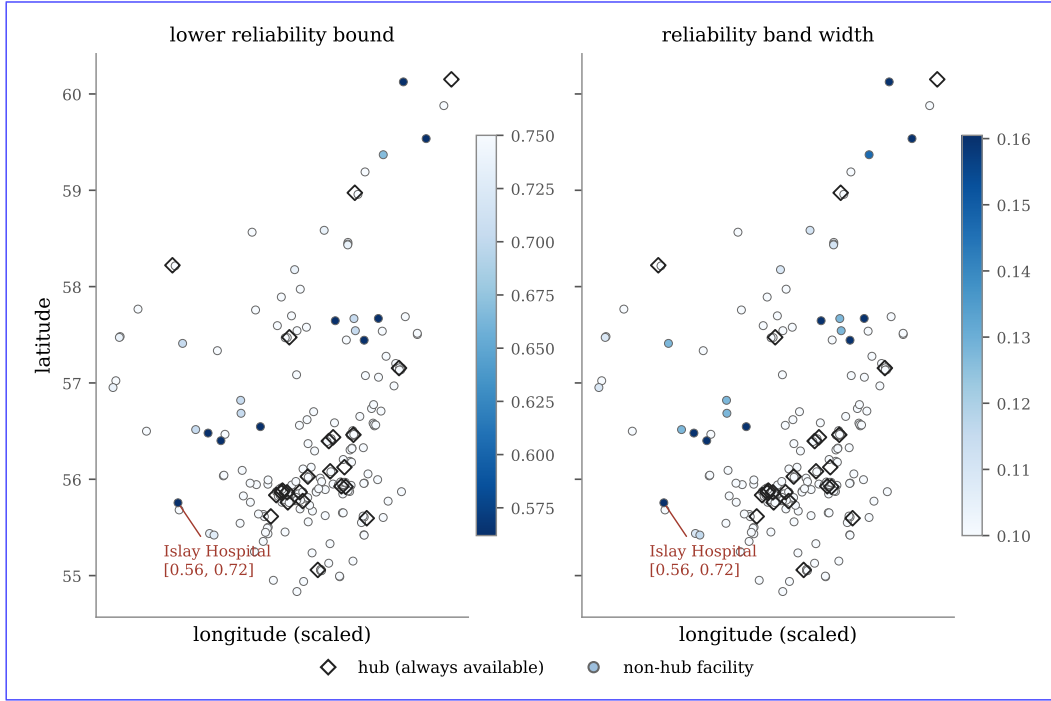


Figure 10: Reliability map of the dense decentralised configuration: each facility coloured by the lower bound of its propagated reachability reliability (left; darker = less certainly reachable) and by the width of its reliability band (right; darker = more uncertainty). Hubs (diamonds) are assumed always available. Uncertainty concentrates at island and far-northern facilities; the worst-served facility, Islay Hospital, is bounded at $[0.56, 0.72]$.

For tree-structured networks and for DAGs with shallow or moderately nested diamonds, the algorithm remains efficient because most node updates are local and independent, and only a limited number of reconvergent subgraphs require conditional evaluation. However, when reconvergent structures become deeply nested, the number of conditioning states grows exponentially. In such cases, progressive decomposition reduces repeated evaluation but cannot eliminate the intrinsic complexity of exact inference. The smallest circuit, *c17* (11 nodes), is trivial (conditioning width 1) and was fully validated across all three input types, including a p-box propagation whose cost matched the prediction of Section 5.3.2's realised-work relationship (predicted 1.3–2.3 s at 50 discretisation levels; measured 1.41 s). The next three circuits tested, *c432* (196 nodes), *c499* (243 nodes), and *c880* (443 nodes), reach conditioning widths of 67, 41, and 50 respectively, an order of magnitude beyond any

other network in this corpus, placing exact propagation on these circuits well outside a practical budget; two further circuits, *c1355* and *c1908* (587 and 913 nodes), exceed the memory available on a 16 GB workstation during identification itself.

The case studies show that three factors are particularly important: **Nesting depth.** Deeply nested diamonds generate repeated conditional branching and are the dominant driver of computational cost. **Conditioning-set size.** Diamonds with many shared upstream variables require a larger number of conditional states to be evaluated. **Context-sensitive reuse.** When the effective input state of a resolved diamond varies across outer conditioning contexts, This is not a boundary specific to IPA. El Fattah and Dechter [39], studying the same circuits with classical tree-clustering and cutset conditioning, the general method IPA specialises (Section 4.3), independently report clique and cutset sizes for *c432* that they describe as “clearly not feasible” for any exact structural method, and comparably extreme figures for the larger circuits in the suite. Decision-diagram size is governed by the same structural width, so a well-ordered decision diagram meets the same wall on these circuits; it does not resolve it.

5.6. Limitations and Future Work

Exact point-valued reliability. For point-valued reliability, IPA has no asymptotic advantage over a well-ordered decision diagram: both are exponential in the network’s structural width (the conditioning width here, the pathwidth for the diagram), reflecting the #P-hardness of network reliability, and across the corpus of Section 5.2 neither dominates. Its distinctive value instead lies in imprecise propagation. Interval results are exact at machine precision, and the reuse of stored supernode maps becomes less effective.

The current decomposition framework naturally identifies the specific subgraphs responsible for computational intractability. Rather than approximating the entire network, future work could apply approximation selectively to deeply nested diamonds while maintaining exact computation elsewhere. This would localise approximation error to the structurally difficult parts of the graph and preserve exactness on the remaining network speed comparison of Section 5.3.1 is scoped to one-shot queries. Distributional bounds are sound, but their tightness is structure-dependent, tight for high-reliability or weakly reconvergent systems and conservative for strongly reconvergent systems with broadly uncertain components, and their cost grows with the discretisation level, which confines full distributional propagation to modest

networks at present; for larger networks, interval propagation combined with sampling is the practical recourse. The tighter positive-dependence operator is validated empirically but not proven sound in general; every guaranteed claim in this paper uses the proven Fréchet operator instead.

Practical range. IPA is exponential in width and intended for the sparse-to-moderate regime, a ceiling shared with every exact method: Section 5.5 located it on a real published circuit family, and the applied case study locates it on a real infrastructure network, where a conditioning requirement of 16, reached at sixteen provisioned alternate routes per location, remains practical (tens of seconds), while unrestricted connectivity makes exact diamond identification itself exceed a 16 GB memory budget before propagation can begin. Imprecise propagation does not relax this boundary: interval and p-box propagation use the same conditioning depth as exact point propagation, and only a change to the network design itself alters the cost. For networks beyond the practical range, a depth-limited hybrid suggests itself: condition exactly to a chosen depth and, beyond it, combine reconvergent contributions without conditioning, which for interval inputs gives a rigorous outer bound (wider, but sound) and for p-box inputs would use the Fréchet combination for the unconditioned remainder. This hybrid is proposed, not implemented or evaluated here; the method reported in this paper does not degrade exactness anywhere.

Structural scope. The method addresses directed acyclic dependency structures. The applied case study’s networks are directed by the hub-and-spoke supply logic of their source design; general cyclic or bidirectional dependency networks, feedback loops and return flows, are outside the present scope, and their principled treatment, for example by unrolling or by strongly-connected-component condensation, is future work. ~~In the present implementation, diamonds are resolved progressively during evaluation. A useful extension would be to perform a more global preprocessing step to identify the full reconvergent structure of the graph in advance and optimise the order in which diamonds are decomposed. This may reduce repeated work and improve tractability in dense dependency networks.~~

6. Conclusions

~~An-~~

This paper has presented the Information Propagation Algorithm (IPA),
an exact reliability analysis method for directed acyclic process networks with
reconvergent dependency structures ~~named Information Propagation Algorithm~~
(IPA) ~~has been proposed~~. The method computes source-to-node reliability
by propagating ~~local~~-reachability probabilities in topological order ~~and~~, re-
solves dependent incoming signals ~~through conditional decomposition of by~~
~~conditioning on separating sets of shared ancestors, and replaces resolved~~
~~fork-join diamond structures. Once resolved, these structures are replaced~~
~~by substructures with~~ statistically equivalent supernodes, ~~allowing the graph~~
~~to be progressively simplified while preserving exactness. This reduces the~~
~~complexity from $O(2^{\sum c_i} \times \prod d_i)$ without decomposition to $O(\sum 2^{c_i})$ with~~
~~decomposition for subsequent belief updates on the same diamond structure.~~
IPA is a specialisation of cutset conditioning to source-to-node reachability:
its per-instance cost is bounded by $\sum_d 2^{|C_d|}$ over the diamonds resolved,
~~computable before enumeration begins, and an independent-substructure~~
~~factorisation reduces fan-in conditioning from exponential to linear where~~
~~independence permits.~~

~~The proposed method is therefore not only an exact message-passing~~
~~formulation for DAG reliability, but a structured way of resolving reconvergent~~
~~dependencies without resorting to global path enumeration. IPA provides an~~
~~exact and efficient tool for a broad class of acyclic process systems, while also~~
~~identifying the structural regimes in which approximation methods will still~~
~~be necessary.~~

~~Two case studies were presented to validate~~ Exactness was verified against
~~an independent exact method, a reduced-ordered binary decision diagram~~
~~with dynamic variable reordering, across 129 random and mutated networks,~~
~~six topological families, and several structured and real-world networks, with~~
~~the worst observed disagreement at floating-point precision. The same comparison~~
~~shows that~~ IPA's effectiveness and accuracy. ~~The comparison against the~~
~~dPrPm, another message passing algorithm for signal propagation in DAGs,~~
~~shows a speed-up in the calculation larger than two order of magnitude while~~
~~computed the same result as exact path enumeration. The second case study~~
~~applied IPA to six Pareto-optimal~~ cost is governed by the same structural
width parameter as the diagram's size, and neither method dominates: for
exact point-valued reliability IPA is competitive with, not superior to, a
well-ordered decision diagram. Its distinguishing capability is the native
propagation of imprecise component reliabilities. Interval-valued inputs yield
the exact reliability range at machine precision, faster for one-shot queries

than the decision-diagram route to the same interval on every family tested. Probability-box inputs yield guaranteed distributional bounds through a conditioning operator grounded in the Fréchet–Hoeffding inequalities, which certify requirement-violation probabilities from a single propagation, a result no point-valued exact method produces analytically.

An applied case study of a medical drone logistics network configurations spanning a range of diamond complexity with up to 277 diamonds and a nesting depth of 18. The drone-network case study indicates that networks with moderate numbers of nodes and edges may remain highly tractable if reconvergent structures are shallow (depth ≤ 12), whereas significantly smaller networks may already become impractical for exact analysis if reconvergent dependencies are deeply nested (depth ≥ 18).

Overall, the results suggest that for Scotland, with every input traced to the source design study and one explicitly flagged extension, illustrates the engineering use of the method: propagated reliability bounds identify the facilities whose delivery reachability is most uncertain under the stated input uncertainty, and a sweep of a real redundancy design parameter locates the practical boundary of exact reliability analysis in large DAGs on the network. Within that boundary, IPA completes full exact propagation in under 25 seconds at a redundancy level where the decision-diagram alternative did not complete within a practical budget. Exact reliability analysis of large acyclic networks is feasible whenever dependency-inducing substructures remain sufficiently localised. This makes IPA particularly relevant for infrastructure design, performance assessment, and decision-support problems in systems that can be represented as multi-source acyclic process networks, and this boundary can be measured and expressed in design terms.

Although the IPA is currently formulated for binary node and edge states in directed acyclic graphs the approach can be generalised to handle cases with Future work includes a soundness proof for the tighter positive-dependence conditioning operator, the depth-limited hybrid scheme proposed for networks beyond the exact range, extension to multi-state components, temporally varying reliabilities, and explicitly modelled common-cause failures beyond structural path sharing. Future research directions may include developing optimization algorithms that pre-decompose the full network before belief propagation as well as further parallelization opportunities beyond topological layer processing, particularly during diamond decomposition when nested diamonds are not interconnected, and a principled treatment of cyclic dependency structures.

References

- [1] H. George-Williams, T. V. Santhosh, E. Patelli, Simulation Methods for the Analysis of Complex Systems, 2021. doi:10.1007/978-3-030-83640-5_7.
URL https://link.springer.com/chapter/10.1007/978-3-030-83640-5_7
- [2] T. Ohiani, E. Patelli, The Information Propagation Method for Efficient Network Reliability Analysis, in: 2023 7th International Conference on System Reliability and Safety (ICSRS), 2023, pp. 580–584. doi:10.1109/ICSRS59833.2023.10381157.
- [3] R. Billinton, R. N. Allan, Network modelling and evaluation of simple systems, in: R. Billinton, R. N. Allan (Eds.), Reliability Evaluation of Engineering Systems: Concepts and Techniques, Springer US, Boston, MA, 1992, pp. 81–100. doi:10.1007/978-1-4899-0685-4_4.
URL https://doi.org/10.1007/978-1-4899-0685-4_4
- [4] Rausand, M. (2014), Reliability Quantification, in: Reliability of Safety-Critical Systems, John Wiley & Sons, Ltd, 2014, pp. 91–164, section: 5 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118776353.ch5>.
doi:<https://doi.org/10.1002/9781118776353.ch5>.
URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118776353.ch5>
- [5] C. Ebeling, An Introduction to Reliability and Maintainability Engineering: Third Edition, Waveland Press, 2019.
URL <https://books.google.co.uk/books?id=rh2WDwAAQBAJ>
- [6] S. Distefano, A. Puliafito, Dynamic Reliability Block Diagrams VS Dynamic Fault Trees, 2007 Proceedings - Annual Reliability and Maintainability Symposium (2007) 71–76Conference Name: 2007 Proceedings Annual Reliability and Maintainability Symposium ISBN: 9780780397668 9780780397675 Place: Orlando, FL, USA Publisher: IEEE. doi:10.1109/RAMS.2007.328095.
URL <http://ieeexplore.ieee.org/document/4126327/>

- 1577 [7] L. Xing, S. V. Amari, Fault Tree Analysis, in: K. B. Misra (Ed.), Hand-
 1578 book of Performability Engineering, Springer London, London, 2008,
 1579 pp. 595–620. doi:10.1007/978-1-84800-131-2_38.
 1580 URL https://doi.org/10.1007/978-1-84800-131-2_38
- 1581 [8] M. Čepin, Event Tree Analysis, in: Assessment of Power System Relia-
 1582 bility: Methods and Applications, Springer London, London, 2011, pp.
 1583 89–99. doi:10.1007/978-0-85729-688-7_6.
 1584 URL https://doi.org/10.1007/978-0-85729-688-7_6
- 1585 [9] D. J. Sherwin, A. Bossche, Fault trees, event trees and success
 1586 trees, in: The Reliability, Availability and Productiveness of Systems,
 1587 Springer Netherlands, Dordrecht, 1993, pp. 133–161. doi:10.1007/
 1588 978-94-011-1582-7_9.
 1589 URL https://doi.org/10.1007/978-94-011-1582-7_9
- 1590 [10] A. Rauzy, New algorithms for fault trees analysis, Relia-
 1591 bility Engineering & System Safety 40 (3) (1993) 203–211.
 1592 doi:[https://doi.org/10.1016/0951-8320\(93\)90060-C](https://doi.org/10.1016/0951-8320(93)90060-C).
 1593 URL [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/095183209390060C)
 1594 [095183209390060C](https://www.sciencedirect.com/science/article/pii/095183209390060C)
- 1595 [11] Bryant, Graph-Based Algorithms for Boolean Function Manipulation,
 1596 IEEE Transactions on Computers C-35 (8) (1986) 677–691. doi:10.
 1597 1109/TC.1986.1676819.
- 1598 [12] K. S. Brace, R. L. Rudell, R. E. Bryant, Efficient implementation of a
 1599 BDD package, Conference proceedings on 27th ACM/IEEE design au-
 1600 tomation conference - DAC '90 (1990) 40–45Conference Name: Confer-
 1601 ence proceedings ISBN: 9780897913638 Place: Orlando, Florida, United
 1602 States Publisher: ACM Press. doi:10.1145/123186.123222.
 1603 URL <http://portal.acm.org/citation.cfm?doid=123186.123222>
- 1604 [13] R. E. Bryant, C. Meinel, Ordered Binary Decision Diagrams.
 1605 doi:10.1007/978-1-4615-0817-5_11.
 1606 URL [https://link.springer.com/chapter/10.1007/](https://link.springer.com/chapter/10.1007/978-1-4615-0817-5_11)
 1607 [978-1-4615-0817-5_11](https://link.springer.com/chapter/10.1007/978-1-4615-0817-5_11)
- 1608 [14] A. Mosleh, Estimation of Parameters of Common Cause Failure Models.
 1609 doi:10.1007/978-94-017-0629-2_12.

- 1610 URL [https://link.springer.com/chapter/10.1007/](https://link.springer.com/chapter/10.1007/978-94-017-0629-2_12)
1611 978-94-017-0629-2_12
- 1612 [15] T. B. Kumar, O. C. Sekhar, M. Ramamoorthy, Composite power
1613 system reliability evaluation using modified minimal cut set ap-
1614 proach, Alexandria Engineering Journal 57 (4) (2018) 2521–2528.
1615 doi:<https://doi.org/10.1016/j.aej.2017.09.008>.
1616 URL [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/S1110016817302934)
1617 S1110016817302934
- 1618 [16] H. Li, Q. Zhao, A cut/tie set method for reliability evaluation of control
1619 systems, in: Proceedings of the 2005, American Control Conference,
1620 2005., 2005, pp. 1048–1053 vol. 2, iSSN: 2378-5861. doi:10.1109/ACC.
1621 2005.1470099.
- 1622 [17] W. Liu, J. Li, An improved cut-based recursive decomposition al-
1623 gorithm for reliability analysis of networks, Earthquake Engineer-
1624 ing and Engineering Vibration 11 (1) (2012) 1–10. doi:10.1007/
1625 s11803-012-0093-5.
1626 URL <http://link.springer.com/10.1007/s11803-012-0093-5>
- 1627 [18] Y. Kim, W.-H. Kang, Network reliability analysis of complex systems
1628 using a non-simulation-based method, Reliability Engineering & System
1629 Safety 110 (2013) 80–88. doi:10.1016/j.ress.2012.09.012.
1630 URL [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/S0951832012001925)
1631 S0951832012001925
- 1632 [19] G. Fishman, A Comparison of Four Monte Carlo Methods for Esti-
1633 mating the Probability of s-t Connectedness, IEEE Transactions on
1634 Reliability 35 (2) (1986) 145–155. doi:10.1109/TR.1986.4335388.
1635 URL [https://www.scopus.com/inward/record.uri?eid=2-s2.](https://www.scopus.com/inward/record.uri?eid=2-s2.0-0022736325&doi=10.1109%2fTR.1986.4335388&partnerID=40&md5=80d0fbdb617df7c3757577fbd528b559)
1636 0-0022736325&doi=10.1109%2fTR.1986.4335388&partnerID=40&
1637 md5=80d0fbdb617df7c3757577fbd528b559
- 1638 [20] K.-P. Hui, Monte Carlo Network Reliability Ranking Estimation, IEEE
1639 Transactions on Reliability 56 (1) (2007) 50–57. doi:10.1109/TR.2006.
1640 890898.
- 1641 [21] S.-K. Au, J. L. Beck, Estimation of small failure probabil-
1642 ities in high dimensions by subset simulation, Probabilistic

- 1643 Engineering Mechanics 16 (4) (2001) 263–277. doi:[https://doi.org/10.1016/S0266-8920\(01\)00019-4](https://doi.org/10.1016/S0266-8920(01)00019-4).
 1644
 1645 URL [https://www.sciencedirect.com/science/article/pii/](https://www.sciencedirect.com/science/article/pii/S0266892001000194)
 1646 [S0266892001000194](https://www.sciencedirect.com/science/article/pii/S0266892001000194)
- 1647 [22] J. S. Yedidia, W. T. Freeman, Y. Weiss, Understanding belief prop-
 1648 agation and its generalizations, in: Exploring Artificial Intelligence in
 1649 the New Millennium, Morgan Kaufmann Publishers Inc., San Francisco,
 1650 CA, USA, 2003, pp. 239–269.
- 1651 [23] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of
 1652 Plausible Inference, Morgan Kaufmann series in representation and rea-
 1653 soning, Elsevier Science, 1988.
 1654 URL <https://books.google.co.uk/books?id=AvNID7LyMusC>
- 1655 [24] Y. Tong, I. Tien, Probability Propagation Method for Re-
 1656 liability Assessment of Acyclic Directed Networks, ASCE-
 1657 ASME Journal of Risk and Uncertainty in Engineering Sys-
 1658 tems, Part A: Civil Engineering 5 (3) (2019) 04019011,
 1659 _eprint: <https://ascelibrary.org/doi/pdf/10.1061/AJRUA6.0001017>.
 1660 doi:10.1061/AJRUA6.0001017.
 1661 URL <https://ascelibrary.org/doi/abs/10.1061/AJRUA6.0001017>
- 1662 [25] K. Murphy, Y. Weiss, M. Jordan, Loopy Belief Propagation for Approxi-
 1663 mate Inference: An Empirical Study, Proceedings of the 15th Conference
 1664 on Uncertainty in Artificial Intelligence (Jan. 2013).
- 1665 [26] S. L. Lauritzen, D. J. Spiegelhalter, Local computations with proba-
 1666 bilities on graphical structures and their application to expert systems,
 1667 Journal of the Royal Statistical Society, Series B 50 (2) (1988) 157–224.
- 1668 [27] S. Ferson, V. Kreinovich, L. Ginzburg, D. S. Myers, K. Sentz, Con-
 1669 structing probability boxes and Dempster–Shafer structures, Tech.
 1670 Rep. SAND2002-4015, Sandia National Laboratories, Albuquerque, NM
 1671 (2003).
- 1672 [28] R. C. Williamson, T. Downs, Probabilistic arithmetic I: Numerical meth-
 1673 ods for calculating convolutions and dependency bounds, International
 1674 Journal of Approximate Reasoning 4 (2) (1990) 89–158.

- 1675 [29] C. Jacob, D. Dubois, J. Cardoso, Uncertainty handling in quantita-
 1676 tive bdd-based fault-tree analysis by interval computation, in: Scal-
 1677 able Uncertainty Management (SUM 2011), Vol. 6929 of Lecture Notes
 1678 in Computer Science, Springer, 2011, pp. 205–218. doi:10.1007/
 1679 978-3-642-23963-2_17.
- 1680 [30] A. M. Imakhlaf, Y. Hou, M. Sallak, Extending BDD-based interval
 1681 computation to non-coherent fault trees using belief functions, IFAC-
 1682 PapersOnLine 50 (1) (2017) 12243–12248.
- 1683 [31] L. G. Valiant, The complexity of enumeration and reliability problems,
 1684 SIAM Journal on Computing 8 (3) (1979) 410–421.
- 1685 [32] M. O. Ball, Complexity of network reliability com-
 1686 putations, Networks 10 (2) (1980) 153–165, _eprint:
 1687 <https://onlinelibrary.wiley.com/doi/pdf/10.1002/net.3230100206>.
 1688 doi:<https://doi.org/10.1002/net.3230100206>.
 1689 URL [https://onlinelibrary.wiley.com/doi/abs/10.1002/net.](https://onlinelibrary.wiley.com/doi/abs/10.1002/net.3230100206)
 1690 [3230100206](https://onlinelibrary.wiley.com/doi/abs/10.1002/net.3230100206)
- 1691 [33] D. D. Mauá, F. G. Cozman, Thirty years of credal networks: Specifica-
 1692 tion, algorithms and complexity, International Journal of Approximate
 1693 Reasoning 126 (2020) 133–157.
- 1694 [34] E. Fagioli, M. Zaffalon, 2U: An exact interval propagation algorithm
 1695 for polytrees with binary variables, Artificial Intelligence 106 (1) (1998)
 1696 77–107.
- 1697 [35] A. Gray, Probabilityboundsanalysis.jl: Probability bounds analysis in
 1698 julia, accessed on April 21, 2026 (2024).
 1699 URL [https://github.com/AnderGray/](https://github.com/AnderGray/ProbabilityBoundsAnalysis.jl)
 1700 [ProbabilityBoundsAnalysis.jl](https://github.com/AnderGray/ProbabilityBoundsAnalysis.jl)
- 1701 [36] F. Somenzi, CUDD: CU decision diagram package, University of Col-
 1702 orado Boulder.
 1703 URL <https://github.com/ivmai/cudd>
- 1704 [37] D. Jones, G. Filippi, S. Basu, B. Parsonage, E. Patelli, C. Maddock,
 1705 M. Vasile, M. Fossati, Conceptual design of a medical drone logistics
 1706 network for Scotland, International Journal of Logistics Research and
 1707 ApplicationsSubmitted; update publication status at proof stage (2025).

- 1708 [38] F. Brglez, H. Fujiwara, A neutral netlist of 10 combinational benchmark
1709 circuits and a target translator in FORTRAN, in: Proceedings of the
1710 IEEE International Symposium on Circuits and Systems (ISCAS), 1985,
1711 pp. 695–698.
- 1712 [39] Y. El Fattah, R. Dechter, Diagnosing tree-decomposable circuits, in:
1713 Proceedings of the Twelfth Conference on Uncertainty in Artificial In-
1714 telligence (UAI), 1996, pp. 244–251.

1715 Acknowledgements

1716 This work was supported by the Nuclear Decommissioning Authority
1717 (NDA) in conjunction with the [UK](#) National Nuclear Laboratory ([UKNNL](#)
1718) through the NDA Bursary Scheme (project number NNL/UA/118).

1719 Data and Software Availability

1720 The implementation of the Information Propagation Algorithm (IPA) ~~;~~
1721 ~~together with the benchmark datasets and scripts~~ is released as the open-source
1722 [Julia package `InformationPropagationAnalysis.jl`, registered in the Julia](#)
1723 [General package registry and available at `https://github.com/Temi-Tory/`](#)
1724 [`InformationPropagationAnalysis.jl`. The benchmark networks, validation](#)
1725 [scripts, and result artefacts](#) required to reproduce ~~the case studies, is available~~
1726 ~~at:-~~
1727 [every table and figure in this manuscript are deposited at `https://`](#)
1728 [doi.org/10.5281/zenodo.22821307 under an MIT licence, together with](#)
1729 [a README mapping each artefact to the manuscript table or figure it](#)
1730 [supports.](#)